

BIRIBÁ: SISTEMA WEB PARA A GESTÃO DO PROGRAMA DE
AQUISIÇÃO DE ALIMENTOS

Wilson Albuquerque Ramos

Projeto de Graduação apresentado ao Curso de Engenharia Eletrônica e de Computação da Escola Politécnica, Universidade Federal do Rio de Janeiro, como parte dos requisitos necessários à obtenção do título de Engenheiro.

Orientador: Celso Alexandre Souza de Alvear

Rio de Janeiro

Agosto de 2026



**UNIVERSIDADE FEDERAL DO RIO DE
JANEIRO**

Politécnica
UFRJ

Escola Politécnica

Departamento de Engenharia Eletrônica e de Computação

**BIRIBÁ: SISTEMA WEB PARA A GESTÃO DO PROGRAMA DE
AQUISIÇÃO DE ALIMENTOS**

Wilson Albuquerque Ramos

PROJETO FINAL SUBMETIDO AO CORPO DOCENTE DO DEPARTAMENTO DE ENGENHARIA ELETRÔNICA E DE COMPUTAÇÃO DA ESCOLA POLITÉCNICA DA UNIVERSIDADE FEDERAL DO RIO DE JANEIRO COMO PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO GRAU DE ENGENHEIRO ELETRÔNICO E DE COMPUTAÇÃO.

Autor:

Wilson Albuquerque Ramos

Orientador:

Celso Alexandre Souza de Alvear, D.Sc.

Examinador:

Flávio Luis de Mello, D.Sc.

Examinador:

Felipe Dias de Rezende Machado, M.Sc.

RIO DE JANEIRO, RJ – BRASIL

AGOSTO DE 2026

Albuquerque Ramos, Wilson

Biribá: Sistema web para a gestão do Programa de Aquisição de Alimentos/ Wilson Albuquerque Ramos. – Rio de Janeiro: UFRJ/Escola Politécnica, 2026.

XIII, 79 p.: il.; 29, 7cm.

Orientador: Celso Alexandre Souza de Alvear

Projeto de Graduação – UFRJ/ Escola Politécnica/ Curso de Engenharia Eletrônica e de Computação, 2026.

Referências Bibliográficas: p. 68 – 70.

1. Tecnologia social. 2. Software Livre. 3. Gestão comunitária. 4. Engenharia de software. I. Souza de Alvear, Celso Alexandre. II. Universidade Federal do Rio de Janeiro, UFRJ, Curso de Engenharia Eletrônica e de Computação. III. Biribá: Sistema web para a gestão do Programa de Aquisição de Alimentos.

Agradecimentos

Agradeço aos meus pais, Jocimar e Terezinha, por sempre me mostrarem o caminho dos estudos e minha esposa, Gabrielle, por também me apoiar nessa trajetória.

Agradeço aos amigos que sempre estiveram do meu lado em especial aos que fiz ao longo de toda a graduação, certamente a caminhada foi mais leve com vocês.

Por fim, dedico este trabalho ao povo brasileiro que contribuiu de forma significativa à minha formação e estada nesta Universidade. Este projeto é uma pequena forma de retribuir o investimento e confiança em mim depositados.

RESUMO

BIRIBÁ: SISTEMA WEB PARA A GESTÃO DO PROGRAMA DE AQUISIÇÃO DE ALIMENTOS

Wilson Albuquerque Ramos

Agosto/2026

Orientador: Celso Alexandre Souza de Alvear

Departamento: Engenharia Eletrônica e de Computação

O Programa de Aquisição de Alimentos é um projeto do governo federal que realiza compras diretas de alimentos dos agricultores familiares a preço de mercado sem a necessidade de licitações e distribui para pessoas em situação de insegurança alimentar, assim como a rede socioassistencial, equipamentos públicos e a rede pública de ensino. A fim de realizar a gestão do programa no assentamento PDS - Osvaldo de Oliveira, foi desenvolvido um sistema web que contemple os processos realizados pelos agricultores e o gestor responsável. Para o desenvolvimento do sistema foram usadas abordagens de engenharia de software, como levantamento de requisitos e metodologia ágil. O projeto segue o padrão arquitetural MVC, sendo o módulo responsável pelo processamento e armazenamento de dados e o módulo de interação com o usuário separados em containers. O deploy do projeto foi feito usando o GitLab CI/CD para a geração da imagem e o envio para o servidor de homologação. Foram usados o Docker container para isolar os módulos e o Docker Compose para inicializar a aplicação. Todo o projeto é de código aberto e está disponível no GitLab.

Palavras-Chave: Tecnologia social, Software Livre, Gestão comunitária, Engenharia de software.

ABSTRACT

BIRIBÁ: WEB SYSTEM FOR MANAGING THE FOOD ACQUISITION PROGRAM

Wilson Albuquerque Ramos

August/2026

Advisor: Celso Alexandre Souza de Alvear

Department: Eletronic and Computer Engineering

The Food Acquisition Program is a Brazilian federal government project that purchases food directly from family farmers at market prices, without the need for bidding processes, and distributes it to people experiencing food insecurity, as well as to the social assistance network, public facilities, and the public education system.

In order to manage the program in the PDS Osvaldo de Oliveira settlement, a web system was developed to cover all processes carried out by the farmers and the responsible manager. Software engineering approaches, such as requirements gathering and agile methodology, were used in the development of the system.

The project follows the MVC architectural pattern, with the module responsible for data processing and storage separated from the user interaction module in different containers. The project deployment was carried out using GitLab Actions to generate the image and send it to the staging server.

Docker containers were used to isolate the modules, and Docker Compose was used to start the application. The entire project is open source and available on GitLab.

Palavras-Chave: Social Technology, Free Software, Community Management, Software Engineering.

SIGLAS

ACM	Association for Computing Machinery
API	Interface de Programação de Aplicações
CI/CD	Integração Contínua e Entrega Contínua
CNPJ	Cadastro Nacional da Pessoa Jurídica
CONAB-RJ	Companhia Nacional de Abastecimento do Rio de Janeiro
CPF	Cadastro de Pessoas Físicas
DOCX	Formato de documento do Microsoft Word
DTO	Data Transfer Object
GPL	GNU General Public License
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LTS	Long-Term Support
MIT	Massachusetts Institute of Technology
MUI	Material UI
MVC	Model-View-Controller
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
OTAN	Organização do Tratado do Atlântico Norte
PAA	Programa de Aquisição de Alimentos
PDS	Projeto de Desenvolvimento Sustentável
SOLTEC	Núcleo de Solidariedade Técnica da UFRJ
SQL	Structured Query Language
SWEBOK	Software Engineering Body of Knowledge
TICDEMOS	Tecnologia da Informação, Democracia e Movimentos Sociais
UFRJ	Universidade Federal do Rio de Janeiro
UI	User Interface
UUID	Universally Unique Identifier
VPS	Virtual Private Server
XP	Extreme Programming

Sumário

Lista de Siglas	vii
Lista de Figuras	xi
Lista de Tabelas	xii
Lista de Códigos	xiii
1 Introdução	1
1.1 Tema	1
1.2 Delimitação	1
1.3 Justificativa	2
1.4 Objetivos	2
1.5 Metodologia	2
1.6 Descrição	3
2 Conceitos de Engenharia de Software	4
2.1 Engenharia de Software	4
2.2 Processos de software	5
2.3 Engenharia de Requisitos	6
2.4 Desenvolvimento ágil de software	7
2.5 Software Livre e código aberto	9
3 Arquitetura do sistema	12
3.1 Visão Geral da Arquitetura	12
3.2 Arquitetura Limpa	13
3.3 Camadas da Arquitetura do Sistema	14

3.3.1	Backend	14
3.3.2	Banco de Dados	15
3.3.3	Frontend	15
3.4	Ferramentas de Apoio ao Desenvolvimento	15
3.4.1	Controle de Versão	15
3.4.2	Integração e Entrega Contínua (CI/CD)	16
3.4.3	Containerização e Deploy	16
4	Tecnologias	18
4.1	Tecnologias de desenvolvimento backend	18
4.1.1	Java 17	19
4.1.2	Spring Boot 3	19
4.1.3	Spring Web	20
4.1.4	Spring Data JPA / Hibernate	20
4.1.5	Spring Security	20
4.1.6	JWT	20
4.1.7	PostgreSQL	21
4.1.8	OpenAPI / Swagger	21
4.1.9	Maven	21
4.1.10	JUnit / Mockito	21
4.1.11	pgAdmin	22
4.2	Tecnologias de desenvolvimento frontend	22
4.2.1	React 19	23
4.2.2	Typescript	23
4.2.3	Vite	23
4.2.4	Material UI	23
4.2.5	Emotion	24
4.2.6	Redux Toolkit	24
4.2.7	React Redux	24
4.2.8	React Router DOM	24
4.2.9	Axios	25
4.2.10	ESLint	25
4.3	Tecnologias de Controle de Versão	25

4.3.1	Git	26
4.3.2	GitLab	26
4.4	Tecnologias de Containerização e Deploy	26
4.4.1	Docker	27
4.4.2	GitLab CI/CD	27
5	Implementação	28
5.1	Requisitos do Sistema	28
5.1.1	Casos de uso do sistema	29
5.2	Modelagem do domínio e estrutura dos dados	31
5.3	Arquitetura geral do sistema	33
5.4	Implementação do backend	34
5.4.1	Desafios de implementação do backend	37
5.5	Implementação do frontend	49
5.5.1	Telas implementadas	50
5.5.2	Desafios de implementação do frontend	62
5.6	Integração e Deploy	73
6	Resultados	74
7	Conclusão	75
7.1	Conclusão	75

Lista de Figuras

3.1	Representação da arquitetura modular do sistema, cliente , servidor e banco de dados	13
5.1	Modelo de domínio simplificado do sistema	33
5.2	Estrutura de diretórios do backend	35
5.3	Login do sistema	51
5.4	Seleção de editais	52
5.5	Plano original aba produto por família parcialmente preenchido . . .	53
5.6	Plano original aba produto por família completo	54
5.7	Plano original aba produto por destino parcialmente preenchido . . .	54
5.8	Plano original aba produto por destino completo	55
5.9	Ciclos e Entregas	56
5.10	Relatórios	57
5.11	Detalhes da família	58
5.12	Configuração	58
5.13	Dashboard	59
5.14	Produtos	60
5.15	Coletivos	61
5.16	Usuários	61
5.17	Destinos	62

Lista de Tabelas

5.1	Principais casos de uso do sistema	31
-----	--	----

Lista de Códigos

5.1	Serviço de adicionar produtos ao edital	37
5.2	Serviço para criação de ciclos dentro do sistema	38
5.3	Serviço geração do termo de recebimento	42
5.4	Serviço de login	48
5.5	Async Thunk para adicionar famílias ao edital	62
5.6	Função que calcula quanto falta para um agricultor entregar	65
5.7	Constante que define a timeline dos gráficos do dashboard	68
5.8	Função de login do sistema	70

Capítulo 1

Introdução

1.1 Tema

O Programa de Aquisição de Alimentos (PAA) é uma iniciativa do governo que tem como objetivo incentivar a agricultura familiar por meio de chamadas públicas e promover o acesso a uma alimentação saudável às famílias, sobretudo às que se encontram em situação de vulnerabilidade.

O gerenciamento de uma chamada pública do PAA requer organização e planejamento, visto que, ao longo do edital, são feitas entregas semanais de alimentos que devem ser registradas corretamente. Dito isso, este trabalho tem como objetivo descrever o sistema desenvolvido para gerenciar as entregas realizadas pelos agricultores a cada organização e equipamento de segurança alimentar que atendem à população em situação de insegurança alimentar e vulnerabilidade social.

1.2 Delimitação

O PAA possui seis modalidades no total. O projeto atuará na modalidade Doação Simultânea, que hoje é aplicada no assentamento dos agricultores do Projeto de Desenvolvimento Sustentável (PDS) Osvaldo de Oliveira, localizado no município de Macaé, na região Norte Fluminense.

1.3 Justificativa

Atualmente, a gestão da chamada pública em exercício no assentamento encontra-se centralizada em uma única pessoa, de modo que tarefas relacionadas às entregas semanais realizadas pelas famílias, à geração de comprovantes necessários para os atores do programa e ao monitoramento da chamada sobrecarregam seu trabalho e demandam tempo para serem realizadas.

1.4 Objetivos

Nesse contexto, o sistema tem como objetivo fornecer ao gestor um panorama geral do andamento do edital, padronizando a inserção de dados e sinalizando erros e inconsistências de forma visual, por meio de uma interface intuitiva. Além disso, automatiza a geração de comprovantes para os produtores, bem como da documentação necessária à validação das entregas.

1.5 Metodologia

O levantamento dos requisitos funcionais e não funcionais do sistema foi realizado por meio de reuniões com os coordenadores do PAA e a partir dos feedbacks fornecidos por eles ao longo do processo. Após o mapeamento das funcionalidades necessárias, tornou-se essencial definir as tecnologias a serem utilizadas no desenvolvimento, bem como o tipo de arquitetura a ser adotada.

Optou-se por uma arquitetura modular, garantindo a independência entre backend e frontend. No backend, foi escolhido o Java, uma linguagem consolidada no mercado, reconhecida por sua robustez, estabilidade e ampla comunidade, o que contribui para a manutenção e evolução do sistema. Para o frontend, adotou-se a biblioteca React, com o objetivo de proporcionar melhor performance e uma navegação mais fluida aos usuários. Como sistema gerenciador de banco de dados, utilizou-se o PostgreSQL, por se tratar de uma solução open source, estável, robusta e amplamente empregada em aplicações empresariais.

Todo o projeto foi versionado com o uso do Git e armazenado no GitLab. Cada módulo possui sua própria imagem Docker, armazenada no DockerHub. Para a

integração contínua, geração das imagens e realização do deploy no ambiente de homologação, foi utilizado o GitLab CI/CD. Ressalta-se que todo o projeto é open source e pode ser clonado nos respectivos repositórios.

1.6 Descrição

No capítulo 2, são apresentados conceitos de Engenharia de Software relevantes para a construção do sistema. O capítulo aborda a importância do desenvolvimento de software de forma planejada e orientada à qualidade, discute processos de software, desenvolvimento ágil e apresenta a relação entre Software Livre, Código Aberto e continuidade tecnológica.

No capítulo 3, é discutida a arquitetura do sistema, com ênfase na organização modular da aplicação e na separação entre frontend, backend e banco de dados. Também são apresentadas as ferramentas de apoio ao desenvolvimento. No capítulo 4, são descritas as tecnologias utilizadas no desenvolvimento da plataforma. O capítulo reúne todas as ferramentas adotadas para o desenvolvimento.

No capítulo 5, é apresentada a implementação do sistema Biribá. São descritos todo o processo de desenvolvimento de software, desde o levantamento de requisitos até a implementação. No capítulo 6, são apresentados os resultados obtidos até o momento, destacando o estágio de homologação da aplicação e as funcionalidades implementadas.

Por fim, no capítulo 7, são apresentadas as conclusões do trabalho, retomando os objetivos alcançados juntamente com a contribuição do sistema para a redução de controles manuais e os trabalhos futuros necessários para o amadurecimento da ferramenta.

Capítulo 2

Conceitos de Engenharia de Software

2.1 Engenharia de Software

O software está presente no cotidiano do ser humano desde que os computadores e os sistemas embarcados se baratearam e ficaram mais acessíveis. Desenvolver nos dias de hoje com a chegada da inteligência artificial pode ser fácil e simples, desde que seja para uso pessoal. No entanto, quando falamos de softwares grandes, com demandas e comportamentos particulares, que resolvem problemas complexos, devemos ter ciência de que, para o seu desenvolvimento, deve haver um processo presente. A engenharia é tradicionalmente conhecida como a área que define processos e soluções para problemas complexos. Ou seja, para o desenvolvimento de software profissional, devemos seguir padrões e conceitos da Engenharia de Software. Essa necessidade de tratar o desenvolvimento de software como um processo organizado contribuiu para o surgimento do termo Engenharia de Software, apresentado na conferência da OTAN de 1968. O termo surgiu devido à demanda de que softwares fossem construídos com base em princípios práticos e teóricos, como todo produto de engenharia. Podemos definir Engenharia de Software, de acordo com Sommerville (2018, p. 19), como uma disciplina de engenharia que se preocupa com os aspectos da produção de software, desde sua concepção inicial até sua operação e manutenção. Com o desenvolvimento da área e sua consolidação como campo de conhecimento a IEEE Computer Society em parceria com a ACM, iniciou em 1998

o projeto SWEBOK com o objetivo de organizar e formalizar os principais conhecimentos relacionados à Engenharia de Software. Esse movimento evidencia que a área passou a envolver diferentes dimensões do desenvolvimento, como requisitos, projeto, construção, testes, qualidade, manutenção, processos, métodos, ferramentas, gerência de configuração e gerência de engenharia de software, mostrando que a construção de sistemas exige mais do que apenas a implementação do código. Mesmo com esse avanço na definição de processos, é consenso que não há uma bala de prata quando se trata da construção de um software, visto que cada problema tem uma particularidade e uma demanda, o que se reflete na construção do software. Softwares de caráter crítico, como, por exemplo, o sistema de piloto automático de uma aeronave, não devem ser construídos do mesmo modo que páginas web. Contudo, podemos destacar propriedades que foram elencadas ao longo do tempo para o desenvolvimento de bons softwares, seja qual for o tipo. São elas: planejamento e objetivo claro; os sistemas devem ser resultado de um processo gerenciável e de fácil compreensão. O software deve se comportar do modo esperado e estar disponível sempre que necessário, além de ser seguro, prevenindo ataques externos. É necessário controlar e compreender especificações e requisitos do sistema. Por último, o software deve funcionar de modo eficaz e sempre deve reutilizar recursos que já tenham sido desenvolvidos para não reinventar a roda.

2.2 Processos de software

Os processos de software correspondem às formas pelas quais as atividades de desenvolvimento são organizadas e conduzidas ao longo de um projeto. Enquanto a Engenharia de Software estabelece uma perspectiva mais ampla sobre a produção de sistemas de qualidade, os processos dizem respeito à maneira concreta como o trabalho é estruturado, distribuído e acompanhado na prática, oferecendo referenciais para orientar a execução do projeto, favorecendo a coordenação entre os participantes e tornando o desenvolvimento mais compreensível além de controlável. De modo geral, os processos de software envolvem atividades relacionadas à definição do que será construído, à implementação da solução, à verificação de seu funcionamento e às modificações necessárias após as primeiras entregas. Essas atividades não se

apresentam da mesma forma em todos os projetos, pois variam conforme o contexto, a natureza do sistema e a forma de organização da equipe. Por isso, a noção de processo não deve ser associada a uma sequência rígida e única de etapas, mas a um conjunto de práticas que orienta o desenvolvimento de acordo com determinadas escolhas metodológicas. Entre os modelos de processo mais conhecidos, destaca-se o modelo em cascata, baseado na execução sequencial das fases de desenvolvimento, o que favorece maior formalização e planejamento prévio. Essa abordagem tende a ser mais adequada em situações nas quais o escopo está claramente definido e há baixa expectativa de mudança. Em contrapartida, sua estrutura linear dificulta ajustes ao longo do processo, especialmente em projetos marcados por incertezas. Como alternativa, consolidaram-se abordagens iterativas e ágeis, orientadas por ciclos mais curtos, entregas frequentes e acompanhamento contínuo do trabalho. Desse modo, a escolha do processo depende das características do projeto e do contexto em que ele se insere, não havendo um modelo único capaz de atender adequadamente a todas as situações.

2.3 Engenharia de Requisitos

A engenharia de requisitos corresponde à etapa da Engenharia de Software responsável por compreender, organizar e definir aquilo que o sistema deve realizar. Enquanto a Engenharia de Software trata da construção de sistemas de forma mais ampla, a engenharia de requisitos concentra-se no entendimento das necessidades dos usuários, das características do domínio do problema e das restrições que devem ser consideradas durante o desenvolvimento. Os requisitos podem ser classificados de diferentes formas, como os requisitos de usuário descrevem, em uma linguagem mais próxima dos envolvidos no projeto, as necessidades que o sistema deve atender. Já os requisitos de sistema apresentam essas necessidades de maneira mais detalhada, servindo como base para o desenvolvimento, os testes e a validação da solução. Temos também, os requisitos funcionais indicando as funcionalidades que o sistema deve oferecer, enquanto os requisitos não funcionais, que podem ser entendidos como restrições no sistema, estão relacionados a aspectos como segurança, desempenho, usabilidade, disponibilidade e manutenção. O processo de engenharia

de requisitos envolve atividades que transformam uma necessidade inicial em uma descrição mais clara do sistema. Entre essas atividades está a elicitação de requisitos, responsável por compreender o problema que o software pretende resolver, considerando o domínio do negócio, os usuários envolvidos, os processos existentes e as dificuldades encontradas. Após essa descoberta, os requisitos são classificados, organizados e priorizados, permitindo definir o que será desenvolvido primeiro e o que poderá ser tratado em etapas futuras. As histórias de usuário também podem apoiar esse processo, pois descrevem as necessidades a partir da perspectiva de quem utilizará o sistema. Além da elicitação, a especificação de requisitos registra os requisitos de usuário e de sistema em um documento próprio. Esse registro pode ser feito em linguagem natural, facilitando o entendimento dos participantes do projeto, ou por meio de modelos mais estruturados, como casos de uso e descrições das funcionalidades. Em seguida, ocorre a validação dos requisitos, etapa responsável por verificar se aquilo que foi definido representa corretamente as necessidades dos usuários e se não existem ambiguidades, inconsistências ou informações incompletas. Como resultado, tem-se o documento de requisitos de software, também chamado de especificação do sistema, que representa a declaração oficial do que deve ser implementado. Esse documento orienta o desenvolvimento e permite acompanhar o que foi definido, implementado ou alterado ao longo do projeto. Como os requisitos podem mudar conforme o problema é melhor compreendido ou novas necessidades surgem, o gerenciamento de requisitos torna-se essencial para controlar alterações, registrar decisões e manter o sistema alinhado aos objetivos do projeto.

2.4 Desenvolvimento ágil de software

O desenvolvimento ágil de software corresponde a uma forma de organização do trabalho voltada à adaptação contínua, à colaboração entre os participantes do projeto e à entrega frequente de resultados utilizáveis. Diferentemente de abordagens centradas em planejamento rígido e definição prévia de todas as etapas, a perspectiva ágil parte do entendimento de que o desenvolvimento ocorre em contextos dinâmicos, nos quais necessidades, prioridades e restrições podem se modificar ao longo do processo. Por essa razão, valoriza-se a capacidade de ajustar o percurso

do projeto de maneira progressiva, sem perder de vista a coerência técnica e os objetivos da solução. Nessa abordagem, a comunicação entre equipe, usuários e demais envolvidos assume papel central. O desenvolvimento deixa de ser compreendido como mera execução sequencial de atividades previamente fixadas e passa a ser conduzido por ciclos curtos de trabalho, nos quais a construção, a avaliação e o refinamento da solução ocorrem de forma recorrente. Essa dinâmica favorece o alinhamento entre o que está sendo produzido e as necessidades efetivas do contexto de uso, além de permitir correções de rumo ao longo do projeto. Outro aspecto característico do desenvolvimento ágil é sua base iterativa e incremental. Em vez de concentrar a validação apenas ao final do processo, a solução é construída em partes menores, que podem ser implementadas, avaliadas e aperfeiçoadas sucessivamente. Esse modo de organização contribui para a identificação antecipada de problemas, para a incorporação mais rápida de feedback e para a redução de riscos associados a interpretações inadequadas dos requisitos. Ao mesmo tempo, amplia a possibilidade de entregas parciais com valor prático, tornando o processo mais sensível às transformações do ambiente e às demandas que emergem no decorrer do trabalho. Entre as principais formas de operacionalização do desenvolvimento ágil, destacam-se o Scrum, o Kanban e o Extreme Programming (XP). Embora essas abordagens compartilhem princípios comuns, como desenvolvimento incremental, adaptação a mudanças e entregas frequentes, elas apresentam ênfases diferentes dentro do processo de desenvolvimento. O Scrum está mais relacionado à gestão e organização do projeto, enquanto o XP se concentra de forma mais direta nas práticas técnicas de codificação e qualidade do software.

O Scrum pode ser compreendido como um framework voltado à organização do trabalho em ambientes complexos. Sua estrutura se apoia em ciclos curtos, chamados Sprints, nos quais a equipe transforma parte das demandas do produto em incrementos de valor. Nesse processo, temos a figura do Product Owner que organiza e prioriza o trabalho, o Scrum Master apoia a aplicação do framework e a remoção de impedimentos, enquanto os desenvolvedores atuam na construção das entregas. Além disso, momentos como planejamento, reuniões diárias, revisão e retrospectiva favorecem o acompanhamento, a inspeção e a adaptação contínua do projeto. Dessa forma, o Scrum contribui principalmente para estruturar a gestão do

desenvolvimento, permitindo maior visibilidade sobre o andamento das atividades e facilitando ajustes ao longo do processo.

Já o Extreme Programming, conhecido como XP, possui uma ênfase mais técnica e está diretamente relacionado à forma como o software é construído. Enquanto o Scrum organiza o trabalho da equipe e a evolução do produto, o XP propõe práticas voltadas à codificação, aos testes e à melhoria contínua da qualidade do código. Entre essas práticas, destacam-se o desenvolvimento orientado por testes, a refatoração contínua, a integração frequente, a programação em pares e o feedback constante entre desenvolvedores e usuários. O XP busca reduzir erros, melhorar a manutenção do sistema e garantir que as entregas sejam realizadas com maior qualidade técnica. Assim, Scrum e XP podem ser utilizados de forma complementar, pois o primeiro contribui para a gestão do projeto, enquanto o segundo fortalece o processo de implementação. Entretanto, a adoção dessa abordagem não implica ausência de método ou informalidade excessiva. A flexibilidade característica do desenvolvimento ágil exige coordenação, definição de prioridades, comprometimento da equipe e mecanismos contínuos de acompanhamento. Sua efetividade depende, portanto, da capacidade de conciliar adaptação com disciplina técnica, preservando a qualidade da solução ao mesmo tempo em que se responde às mudanças do projeto.

2.5 Software Livre e código aberto

No campo do desenvolvimento de software, a discussão sobre Software Livre e Código Aberto insere-se de forma relevante entre os temas relacionados à produção, à circulação e à evolução de soluções computacionais. Para além dos aspectos estritamente técnicos, esses conceitos envolvem diferentes formas de acesso ao código-fonte, possibilidades de modificação do software e modos de compartilhamento da tecnologia. Nesse sentido, sua compreensão contribui para ampliar o debate sobre como sistemas são desenvolvidos, mantidos e apropriados por diferentes grupos ao longo do tempo.

O Software Livre corresponde da liberdade ao usuário para utilizar, estudar, modificar e redistribuir programas de computador. Assim, sua característica central não está associada à gratuidade, mas à garantia de autonomia sobre o uso e a

transformação da tecnologia. Trata-se, portanto, de uma perspectiva que ultrapassa a dimensão estritamente operacional do software, incorporando também a possibilidade de compartilhamento do conhecimento, adaptação a diferentes contextos e fortalecimento da independência tecnológica. Essa característica torna o Software Livre relevante em contextos nos quais a continuidade, a transparência e a possibilidade de adaptação da solução constituem aspectos importantes. O software deixa de ser entendido apenas como produto técnico fechado e passa a ser concebido como artefato passível de apropriação, modificação e evolução por diferentes atores, de acordo com necessidades específicas e mudanças de contexto.

O Open Source também se baseia na disponibilização do código-fonte e na possibilidade de modificação e redistribuição do software. Contudo, sua ênfase recai principalmente sobre as vantagens práticas desse modelo de desenvolvimento, como colaboração distribuída, aperfeiçoamento contínuo e maior flexibilidade na evolução das soluções. Já a noção de Software Livre atribui centralidade à liberdade dos usuários e ao valor social associado ao compartilhamento da tecnologia. Desse modo, embora os dois conceitos sejam próximos e frequentemente apareçam associados, eles não são equivalentes. Ambos admitem abertura do código e contribuição coletiva, mas diferem quanto ao enfoque que orienta sua formulação. Enquanto o Open Source enfatiza os benefícios técnicos e organizacionais do desenvolvimento aberto, o Software Livre ressalta a autonomia, a circulação do conhecimento e a possibilidade de apropriação social da tecnologia. Essa distinção é relevante porque evidencia que a abertura do software pode ser compreendida tanto como estratégia de desenvolvimento quanto como compromisso com formas mais amplas de uso, adaptação e continuidade das soluções computacionais.

A adoção de Software Livre e de soluções Open Source também está diretamente relacionada ao tipo de licença que regula o uso, a modificação e a redistribuição do software. Essas licenças definem juridicamente as condições sob as quais o código pode ser utilizado por terceiros, influenciando o grau de liberdade, compartilhamento e reaproveitamento permitido. Entre as licenças mais conhecidas, destaca-se a licença MIT, caracterizada por maior permissividade, permitindo reutilização, modificação e redistribuição com poucas restrições. Em contraste, a licença GPL enfatiza a preservação das liberdades associadas ao software, exigindo que versões de-

rivadas mantenham a mesma lógica de abertura do código. Dessa forma, as licenças não apenas regulamentam aspectos legais do software, mas também expressam diferentes entendimentos sobre colaboração, circulação do conhecimento e continuidade tecnológica.

Capítulo 3

Arquitetura do sistema

3.1 Visão Geral da Arquitetura

A arquitetura de um sistema corresponde à forma como seus componentes são organizados e relacionados para atender aos requisitos funcionais e não funcionais da aplicação. No campo da Engenharia de Software, essa definição é importante porque a arquitetura não se restringe à estrutura técnica do sistema, mas influencia diretamente aspectos como manutenção, escalabilidade, evolução e integração com outras soluções. Nesse sentido, a definição arquitetural constitui uma etapa relevante do desenvolvimento, pois estabelece a base sobre a qual o sistema será construído e ampliado ao longo do tempo. Entre as diferentes possibilidades de organização arquitetural, destaca-se a arquitetura modular, caracterizada pela divisão do sistema em partes com responsabilidades relativamente bem delimitadas. Essa abordagem favorece a separação de interesses, reduz o acoplamento entre componentes e facilita a compreensão, a manutenção e a evolução da aplicação. Além disso, a modularidade contribui para que novas funcionalidades possam ser incorporadas sem comprometer de forma significativa o funcionamento das partes já existentes, o que a torna especialmente adequada para sistemas que demandam crescimento progressivo e adaptação a diferentes contextos de uso.

No desenvolvimento de sistemas web, uma forma recorrente de materializar essa organização modular é a divisão entre frontend e backend. A camada de frontend é responsável pela interação com o usuário, concentrando os elementos visuais, os mecanismos de navegação e a apresentação das informações. Já a camada de bac-

kend concentra a lógica de negócio, o controle de acesso aos dados e a comunicação com o banco de dados. Essa separação contribui para distribuir responsabilidades de maneira mais clara, permitindo que interface e processamento interno evoluam com maior independência, ainda que permaneçam articulados no funcionamento da aplicação. De modo complementar, a comunicação entre essas camadas costuma ocorrer por meio de interfaces de programação de aplicações, que permitem a troca estruturada de dados e favorecem a integração entre diferentes componentes e serviços. Em arquiteturas desse tipo, o banco de dados atua como mecanismo de persistência das informações, enquanto ferramentas de versionamento, containerização e automação de processos apoiam o desenvolvimento, a manutenção e a distribuição do sistema. Assim, a arquitetura modular em camadas constitui uma solução adequada para aplicações que exigem clareza estrutural, possibilidade de expansão e integração com outros sistemas. A organização geral dessa arquitetura será apresentada na figura a seguir.



Figura 3.1: Representação da arquitetura modular do sistema, cliente , servidor e banco de dados

3.2 Arquitetura Limpa

A arquitetura limpa é uma abordagem de organização de software que busca separar as responsabilidades do sistema em camadas, reduzindo o acoplamento entre regras de negócio, tecnologias externas e mecanismos de entrada e saída. Dessa forma, a lógica central da aplicação não fica dependente diretamente de frameworks, banco de dados, interfaces gráficas ou protocolos de comunicação. Essa separação contribui para que o sistema seja mais fácil de manter, testar e evoluir, pois alterações em uma tecnologia específica não afetam diretamente as regras principais do domínio. A arquitetura é estruturada em camadas bem definidas, cada uma com responsabilidades específicas e com dependências direcionadas para o núcleo do sistema, garantindo que as regras de negócio permaneçam independentes de detalhes

externos.

Seguindo esses princípios, a camada domain representa o núcleo da aplicação, sendo responsável por conter as entidades, regras de negócio e abstrações fundamentais do sistema, sem depender de tecnologias externas. A camada application atua como intermediária entre o domínio e as demais camadas, coordenando os casos de uso e definindo contratos para a execução das operações do sistema. A camada infrastructure é responsável por implementar detalhes técnicos, como acesso a banco de dados, integração com serviços externos e configurações necessárias para o funcionamento da aplicação. Por último, temos a camada web que corresponde à interface de comunicação com o usuário ou outros sistemas, sendo responsável por receber requisições, encaminhá-las para as camadas internas e retornar as respostas apropriadas.

3.3 Camadas da Arquitetura do Sistema

3.3.1 Backend

O backend corresponde à camada responsável pelo processamento das regras de negócio, pela validação dos dados recebidos, pelo controle de acesso e pela comunicação com os mecanismos de armazenamento e processamento de informações. Essa camada concentra a lógica central do sistema, garantindo que as operações sejam executadas de maneira consistente, segura e de acordo com os requisitos funcionais e não funcionais estabelecidos. Entre suas principais responsabilidades, destacam-se o tratamento e a persistência dos dados, a aplicação de restrições e validações, o gerenciamento de autenticação e autorização, o controle das transações, o tratamento de exceções e a disponibilização de serviços para outras camadas da arquitetura. Além disso, o backend contribui para atributos como integridade, confiabilidade, escalabilidade, segurança e manutenibilidade, constituindo um elemento fundamental para o funcionamento adequado de sistemas computacionais.

3.3.2 Banco de Dados

O banco de dados é o componente responsável pelo armazenamento persistente, pela organização e pela recuperação das informações utilizadas por um sistema. Trata-se de um elemento fundamental em aplicações computacionais, pois fornece a base necessária para o registro, a manutenção e o acesso estruturado aos dados. Além de armazenar informações de forma duradoura, o banco de dados também contribui para a integridade, a consistência, a segurança e a disponibilidade dos dados, permitindo que eles sejam consultados, atualizados e relacionados de maneira eficiente. Dessa forma, esse componente sustenta o funcionamento das demais camadas do sistema, servindo de apoio para o processamento realizado pela lógica da aplicação e para a disponibilização das informações aos usuários.

3.3.3 Frontend

O frontend corresponde à camada de apresentação do sistema, ou seja, à interface por meio da qual os usuários interagem com a aplicação. É nessa camada que se encontram os elementos visuais e interativos que possibilitam o uso do sistema, como telas, formulários, menus, listagens, botões e mecanismos de navegação. Sua principal função é apresentar informações e funcionalidades de forma compreensível, organizada e acessível, permitindo que a interação do usuário com o sistema ocorra de maneira eficiente e adequada ao seu contexto de uso. Além disso, o frontend também contribui para a usabilidade, a clareza na comunicação das informações, a responsividade da interface e a mediação entre as ações do usuário e os serviços disponibilizados pelas demais camadas da arquitetura.

3.4 Ferramentas de Apoio ao Desenvolvimento

3.4.1 Controle de Versão

O controle de versão é uma prática fundamental no desenvolvimento de software, pois permite registrar, organizar e acompanhar as alterações realizadas no código-fonte ao longo do tempo. Por meio desse processo, torna-se possível recuperar versões anteriores, comparar modificações, identificar a autoria das mudanças e

manter um histórico estruturado da evolução do sistema. Além disso, o controle de versão desempenha papel essencial no trabalho colaborativo, uma vez que possibilita que diferentes desenvolvedores atuem simultaneamente sobre o mesmo projeto de forma coordenada, reduzindo conflitos e favorecendo a integração contínua das contribuições. Dessa maneira, essa prática contribui para a rastreabilidade, a segurança, a manutenção e a evolução consistente do software.

3.4.2 Integração e Entrega Contínua (CI/CD)

A integração contínua e a entrega contínua, frequentemente designadas pela sigla CI/CD, constituem práticas voltadas à automação e à padronização de etapas do ciclo de desenvolvimento de software. De modo geral, essas práticas buscam assegurar que alterações no código sejam integradas, verificadas e disponibilizadas de forma frequente e controlada, reduzindo a dependência de procedimentos manuais e tornando o processo de desenvolvimento mais confiável. A integração contínua está associada à incorporação recorrente de mudanças em um repositório compartilhado, normalmente acompanhada da execução automática de testes e validações. Já a entrega contínua se relaciona à preparação do software para que novas versões possam ser disponibilizadas de maneira rápida e consistente. Em conjunto, essas práticas contribuem para a detecção antecipada de falhas, para a melhoria da qualidade do software, para a redução de riscos nas liberações e para o aumento da previsibilidade e da eficiência no processo de desenvolvimento.

3.4.3 Containerização e Deploy

A containerização consiste em empacotar aplicações, bibliotecas, arquivos de configuração e demais dependências em unidades padronizadas de execução, denominadas contêineres. Essa abordagem permite que o software seja executado de forma mais consistente em diferentes ambientes computacionais, reduzindo incompatibilidades entre desenvolvimento, teste e produção. Ao isolar a aplicação e os elementos necessários ao seu funcionamento, a containerização favorece a portabilidade, a padronização e a reprodutibilidade, além de contribuir para a escalabilidade, a eficiência na implantação e a simplificação do gerenciamento dos ambientes. Dessa forma, constitui uma prática amplamente adotada para tornar a distribuição e a

execução de sistemas mais previsíveis, controladas e independentes das particularidades da infraestrutura subjacente.

Capítulo 4

Tecnologias

Para viabilizar a aplicação dos conceitos apresentados nos capítulos anteriores, contamos com uma ampla gama de tecnologias que auxiliam no desenvolvimento de sistemas web. Dito isto, este capítulo tem como objetivo descrever o papel de cada tecnologia na arquitetura da solução utilizada no desenvolvimento do projeto Biribá, destacando suas principais características e como elas se relacionam para compor uma aplicação modular. Para a escolha das tecnologias que serão apresentadas a seguir, foram levados em consideração a ampla adoção do mercado, comunidades consolidadas e ativas. Por último, o fato de serem software livre possibilitando a redução dos custos de licenciamento e favorecendo a independência tecnológica durante o desenvolvimento e a evolução da aplicação.

4.1 Tecnologias de desenvolvimento backend

Quando se trata do desenvolvimento de aplicações empresariais, o ecossistema formado por Java e Spring Boot se destaca devido à sua ampla adoção na construção de sistemas corporativos . O Java compõe a base do desenvolvimento do backend, sendo uma linguagem orientada a objetos capaz de representar as regras de negócio da aplicação. Complementando essa estrutura, o Spring Boot fornece um conjunto de ferramentas que simplificam o desenvolvimento e a organização do sistema. Entre elas, destacam-se o Spring Web, responsável pela camada de comunicação da aplicação, o Spring Data JPA, utilizado para a persistência de dados, e o Spring Security, empregado na implementação dos mecanismos de segurança. Para com-

plementar esse processo, o padrão JWT é utilizado para garantir a autenticação e a autorização das requisições realizadas ao backend. Em conjunto, essas tecnologias constituem a estrutura central da aplicação. Além da estrutura principal, escolhemos tecnologias que complementam esse ecossistema e ampliam suas capacidades. O PostgreSQL se integra naturalmente às soluções baseadas em Spring por meio das camadas de persistência de dados. Para facilitar a administração deste banco de dados, o pgAdmin disponibiliza uma interface gráfica que simplifica atividades de gerenciamento e consulta. No contexto do desenvolvimento, ferramentas como Swagger e JUnit também possuem forte integração com o ecossistema Spring, contribuindo, respectivamente, para a documentação de serviços e para a validação automatizada do comportamento da aplicação. Por fim, o Maven atua como a principal ferramenta de gerenciamento de dependências e automação de tarefas em projetos Java, sendo amplamente utilizado em conjunto com o Spring Boot para organizar bibliotecas externas e padronizar processos de compilação, testes e empacotamento da aplicação.

4.1.1 Java 17

O Java 17 é uma linguagem de programação orientada a objetos amplamente utilizada no desenvolvimento de sistemas corporativos e aplicações de grande porte. Por se tratar de uma versão de suporte de longo prazo (Long-Term Support – LTS), oferece maior estabilidade, segurança e compatibilidade para projetos que demandam manutenção contínua. Sua adoção contribui para a construção de aplicações robustas, portáteis e com amplo suporte do ecossistema de desenvolvimento.

4.1.2 Spring Boot 3

O Spring Boot 3 é um framework voltado à simplificação do desenvolvimento de aplicações baseadas na plataforma Spring. Sua principal proposta é reduzir a complexidade de configuração inicial, permitindo a criação de sistemas com maior agilidade e padronização. Além disso, fornece mecanismos que facilitam a organização modular da aplicação, a integração entre componentes e a configuração automatizada de diversos recursos.

4.1.3 Spring Web

O Spring Web é o módulo responsável pelo desenvolvimento da camada de comunicação da aplicação, especialmente no que se refere à construção de APIs e ao tratamento de requisições e respostas. Ele possibilita a definição de controladores, rotas e mecanismos de entrada e saída de dados, servindo como base para a interação entre clientes e serviços do sistema.

4.1.4 Spring Data JPA / Hibernate

O Spring Data JPA é uma tecnologia que simplifica o acesso e a manipulação de dados em aplicações Java, oferecendo uma abstração para operações de persistência. Em conjunto com o Hibernate, que atua como implementação do mapeamento objeto-relacional, permite relacionar entidades do sistema com tabelas do banco de dados. Essa combinação reduz a necessidade de escrita manual de consultas básicas, favorecendo a produtividade, organização e manutenção do código.

4.1.5 Spring Security

O Spring Security é um framework voltado à segurança de aplicações, oferecendo mecanismos para autenticação, autorização e proteção de recursos. Seu uso permite controlar o acesso às funcionalidades do sistema com base em perfis, permissões e regras previamente definidas. Dessa forma, contribui para reforçar a proteção da aplicação contra acessos indevidos e para estruturar de maneira consistente as políticas de segurança adotadas.

4.1.6 JWT

O JWT (JSON Web Token) é um padrão utilizado para representação segura de informações entre partes, sendo amplamente empregado em processos de autenticação e autorização. No contexto de sistemas distribuídos, ele permite que informações sobre a identidade do usuário e suas permissões sejam transmitidas de forma assinada, favorecendo a verificação de acesso sem a necessidade de armazenamento contínuo de sessão no servidor.

4.1.7 PostgreSQL

O PostgreSQL é um sistema gerenciador de banco de dados relacional de código aberto, reconhecido por sua confiabilidade, desempenho e conformidade com padrões SQL. Ele oferece recursos avançados para armazenamento, consulta, integridade e segurança dos dados, sendo amplamente utilizado em aplicações que exigem consistência e escalabilidade na persistência das informações.

4.1.8 OpenAPI / Swagger

O OpenAPI é uma especificação voltada à descrição padronizada de interfaces de programação de aplicações (Application Programming Interfaces – APIs). O Swagger, por sua vez, corresponde a um conjunto de ferramentas que possibilita visualizar, documentar e testar APIs com base nessa especificação. O uso dessas tecnologias favorece a clareza na definição dos contratos da aplicação, facilita a comunicação entre equipes e contribui para a validação e o consumo dos serviços disponibilizados.

4.1.9 Maven

O Maven é uma ferramenta de automação e gerenciamento de projetos utilizada principalmente em aplicações Java. Sua função envolve o controle de dependências, a padronização da estrutura do projeto e a automação de tarefas como compilação, testes e empacotamento. Dessa maneira, contribui para a organização do processo de desenvolvimento e para a reprodutibilidade da construção do software.

4.1.10 JUnit / Mockito

O JUnit é um framework amplamente utilizado para a elaboração e execução de testes automatizados em aplicações Java, especialmente testes unitários. O Mockito, por sua vez, complementa esse processo ao permitir a simulação de comportamentos de objetos e dependências externas. Em conjunto, essas ferramentas contribuem para a verificação do funcionamento do código, para o isolamento de componentes durante os testes e para o aumento da confiabilidade da aplicação.

4.1.11 pgAdmin

O pgAdmin é uma ferramenta gráfica de administração e gerenciamento do PostgreSQL. Sua utilização facilita atividades como visualização de tabelas, execução de consultas, monitoramento do banco e administração de estruturas de dados. Dessa forma, serve como apoio ao desenvolvimento e à manutenção do banco de dados, tornando mais acessíveis operações que poderiam ser realizadas apenas por linha de comando.

4.2 Tecnologias de desenvolvimento frontend

No desenvolvimento da camada de frontend, o ecossistema React se destaca pela ampla utilização na construção de interfaces web modernas e pela flexibilidade oferecida por sua arquitetura baseada em componentes. O React constitui a base da interface da aplicação, permitindo a construção de elementos reutilizáveis e facilitando a organização do código. Complementando essa estrutura, o TypeScript adiciona mecanismos de tipagem estática que contribuem para a segurança e a manutenção do sistema. Para apoiar o processo de desenvolvimento, o Vite é utilizado como ferramenta de construção da aplicação, oferecendo um ambiente de desenvolvimento ágil e eficiente. Complementando a estrutura base do frontend temos, o Material UI e o Emotion atuando na construção e estilização da interface, fornecendo recursos que favorecem a criação de componentes consistentes e a padronização visual da aplicação. Para o gerenciamento de estado global, o React Redux e o Redux Toolkit oferecem mecanismos que facilitam o compartilhamento e a sincronização de informações entre diferentes componentes. No contexto da navegação, o React Router DOM permite a organização das rotas e das diferentes visualizações do sistema, enquanto o Axios viabiliza a comunicação entre o frontend e os serviços disponibilizados pelo backend. Por fim, o ESLint integra-se ao processo de desenvolvimento auxiliando na padronização do código-fonte e na identificação de problemas que possam comprometer sua qualidade e manutenção.

4.2.1 React 19

O React 19 é uma biblioteca JavaScript voltada à construção de interfaces de usuário baseadas em componentes reutilizáveis. Sua abordagem permite dividir a interface em partes independentes, facilitando a organização do código, a manutenção e a evolução da aplicação. Além disso, o React favorece a criação de interfaces dinâmicas e interativas, adequadas ao desenvolvimento de sistemas modernos.

4.2.2 Typescript

O Typescript é uma linguagem que transpila o JavaScript por meio da adição de tipagem estática e outros recursos voltados à maior segurança no desenvolvimento. Sua utilização contribui para a detecção antecipada de erros, para a melhor documentação do código e para a maior previsibilidade no comportamento da aplicação. Dessa forma, favorece a manutenção e a escalabilidade de projetos de maior porte.

4.2.3 Vite

O Vite é uma ferramenta de construção e execução de aplicações frontend que se destaca pela rapidez no ambiente de desenvolvimento e pela simplicidade de configuração. Ele oferece suporte eficiente ao empacotamento do projeto, à atualização instantânea durante a codificação e à preparação da aplicação para produção. Seu uso contribui para tornar o processo de desenvolvimento mais ágil e produtivo.

4.2.4 Material UI

O Material UI (MUI) é uma biblioteca de componentes visuais para aplicações React, baseada nos princípios do Material Design. Ela oferece um conjunto de elementos de interface prontos, como botões, tabelas, campos de formulário, menus e diálogos, permitindo a construção de interfaces consistentes, responsivas e visualmente padronizadas. Seu uso contribui para acelerar o desenvolvimento e melhorar a usabilidade da aplicação.

4.2.5 Emotion

O Emotion é uma biblioteca voltada à estilização de interfaces em aplicações JavaScript e React. Ela permite definir estilos diretamente nos componentes, com flexibilidade para composição, reutilização e manutenção do design da aplicação. Essa abordagem favorece a organização dos estilos e a integração entre lógica e apresentação no desenvolvimento da interface.

4.2.6 Redux Toolkit

O Redux Toolkit é uma ferramenta destinada ao gerenciamento de estado global em aplicações frontend. Sua principal finalidade é simplificar o uso da arquitetura Redux, reduzindo a complexidade de configuração e de manipulação do estado compartilhado entre diferentes partes da aplicação. Dessa forma, contribui para a centralização das informações, para a previsibilidade do fluxo de dados e para a melhor organização do código.

4.2.7 React Redux

O React Redux é a biblioteca responsável por integrar o Redux às aplicações desenvolvidas com React. Ela fornece mecanismos para conectar componentes ao estado global da aplicação e para despachar ações que atualizam os dados compartilhados. Seu uso permite que a interface reaja de forma consistente às mudanças de estado, favorecendo a sincronização entre diferentes componentes.

4.2.8 React Router DOM

O React Router DOM é uma biblioteca utilizada para gerenciar a navegação entre diferentes páginas ou visualizações em aplicações React. Ela possibilita definir rotas, associar componentes a caminhos específicos e controlar a transição entre telas sem a necessidade de recarregamento completo da página. Dessa forma, contribui para a organização da navegação e para uma experiência de uso mais fluida.

4.2.9 Axios

O Axios é uma biblioteca utilizada para realizar requisições HTTP em aplicações JavaScript. No contexto do frontend, sua principal função é possibilitar a comunicação com serviços externos, como APIs responsáveis pelo fornecimento e recebimento de dados. Seu uso facilita o envio de requisições, o tratamento de respostas e o gerenciamento de erros na comunicação entre a interface e o backend.

4.2.10 ESLint

O ESLint é uma ferramenta de análise estática de código voltada à identificação de problemas de sintaxe, inconsistências e más práticas em projetos JavaScript e TypeScript. Sua utilização contribui para a padronização do código-fonte, para a melhoria da legibilidade e para a prevenção de erros durante o desenvolvimento. Assim, auxilia na manutenção da qualidade e da consistência do projeto.

4.3 Tecnologias de Controle de Versão

Ferramentas de controle de versão desempenham um papel fundamental na organização e no acompanhamento das alterações realizadas ao longo do ciclo de vida de um projeto. Entre as soluções disponíveis, o Git se destaca como um dos sistemas de controle de versão mais utilizado no mundo, permitindo registrar modificações no código-fonte, recuperar versões anteriores e gerenciar diferentes linhas de desenvolvimento de forma distribuída. Sua utilização contribui para a rastreabilidade das mudanças, para a segurança das informações e para a colaboração entre os membros da equipe de desenvolvimento. Por sua vez, o GitLab fornece uma plataforma integrada para hospedagem e gerenciamento de repositórios Git. Além de centralizar o armazenamento do código-fonte, a ferramenta disponibiliza recursos voltados ao desenvolvimento colaborativo, como controle de acesso, revisão de código, gerenciamento de atividades e acompanhamento do progresso do projeto. Dessa forma, a combinação entre Git e GitLab possibilita um processo de desenvolvimento mais organizado, favorecendo o controle das alterações, a colaboração entre desenvolvedores e a manutenção contínua do software.

4.3.1 Git

O Git é um sistema de controle de versão distribuído amplamente utilizado no desenvolvimento de software para registrar, organizar e acompanhar as alterações realizadas no código-fonte ao longo do tempo. Por meio dele, torna-se possível manter um histórico detalhado das modificações, recuperar versões anteriores, criar ramificações independentes para desenvolvimento de funcionalidades e integrar mudanças de diferentes colaboradores de forma controlada. Sua utilização contribui para a rastreabilidade das alterações, para a segurança no processo de desenvolvimento e para a manutenção evolutiva do software.

4.3.2 GitLab

O GitLab é uma plataforma de hospedagem e gerenciamento de repositórios baseada no Git, que oferece recursos voltados ao desenvolvimento colaborativo e à automação de processos. Além de permitir o armazenamento centralizado do código-fonte, a ferramenta disponibiliza mecanismos para controle de acesso, revisão de código, gerenciamento de issues, acompanhamento de atividades e integração com fluxos de integração contínua e entrega contínua. Dessa forma, o GitLab amplia as possibilidades de organização do projeto, de colaboração entre os membros da equipe e de controle do ciclo de desenvolvimento de software.

4.4 Tecnologias de Containerização e Deploy

No âmbito da disponibilização de aplicações, tecnologias de containerização e automação de processos desempenham um papel importante na padronização dos ambientes e na redução de atividades manuais durante o ciclo de desenvolvimento. Entre as soluções mais utilizadas, o Docker se destaca por permitir o empacotamento da aplicação e de suas dependências em contêineres, garantindo maior consistência entre os ambientes de desenvolvimento, teste e produção, favorecendo a portabilidade, a reprodutibilidade e a escalabilidade das aplicações. Em conjunto, o GitLab CI/CD complementa essa abordagem ao fornecer mecanismos para automatizar atividades relacionadas à integração contínua e à entrega contínua, permitindo a execução de tarefas como compilação, testes, validações e implantação da aplicação.

sempre que alterações são realizadas no repositório. Dessa forma, a integração entre Docker e GitLab CI/CD possibilita a construção de um processo de entrega mais confiável, padronizado e eficiente, reduzindo intervenções manuais e favorecendo a disponibilização contínua de novas versões do software.

4.4.1 Docker

O Docker é uma plataforma voltada à containerização de aplicações, permitindo empacotar o software e suas dependências em unidades isoladas de execução, denominadas contêineres. No contexto de integração contínua e entrega contínua, seu uso contribui para a padronização dos ambientes de desenvolvimento, teste e implantação, reduzindo inconsistências entre diferentes etapas do ciclo de vida do software. Além disso, o Docker favorece a reprodutibilidade, a portabilidade e a escalabilidade da aplicação, tornando o processo de construção, validação e disponibilização de novas versões mais previsível e controlado.

4.4.2 GitLab CI/CD

O GitLab CI/CD corresponde ao conjunto de mecanismos de automação disponibilizados pela plataforma GitLab para apoiar práticas de integração contínua e entrega contínua. Por meio dessa abordagem, é possível definir fluxos automatizados para executar etapas como compilação, testes, validações e processos de implantação sempre que alterações são realizadas no repositório. Sua utilização contribui para reduzir atividades manuais, aumentar a confiabilidade do processo de desenvolvimento e permitir que a equipe acompanhe de forma mais sistemática a qualidade e a evolução do software. Dessa forma, o GitLab CI/CD fortalece a automação do ciclo de desenvolvimento e favorece a disponibilização mais rápida e consistente de novas versões da aplicação.

Capítulo 5

Implementação

O desenvolvimento do projeto teve início no segundo semestre de 2024, no âmbito da disciplina Software Livre e Metodologias Participativas. Após o término da disciplina, o projeto passou a integrar as atividades do projeto de extensão Tecnologia da Informação, Democracia e Movimentos Sociais (TICDEMOS), vinculado ao SOL-TEC/UFRJ. Desde o início, a proposta contou com o acompanhamento da antiga gestora do PAA no PDS Osvaldo de Oliveira, que contribuiu com sua experiência para a modelagem do sistema e para a definição dos principais casos de uso. Ao longo do desenvolvimento, a proposta passou por mudanças em sua organização técnica e funcional. Inicialmente, o sistema estava voltado para um único assentamento e com criação de uma base mínima que permitisse cadastrar usuários, famílias, produtos, ciclos, destinos e editais. Em 2025, a proposta foi apresentada à equipe da Companhia Nacional de Abastecimento do Rio de Janeiro (CONAB-RJ). Após a reunião, vimos que além do cadastro das informações essenciais do edital deveríamos contemplar o cadastro de coletivos, um painel de acompanhamento que mostrasse de forma clara o andamento do projeto e a geração de relatórios que apoiam a comprovação das entregas, visualizando valores a receber pelas famílias agricultoras, além de, facilitar a consolidação dos dados de entregas pelo coordenador.

5.1 Requisitos do Sistema

O levantamento dos requisitos do sistema foi realizado a partir de duas frentes diferentes, a primeira através de diálogo com a gestora do PAA no assentamento para

entender seu dia-a-dia e como funcionava o diálogo com os agricultores e a segunda da análise baseado em planilhas no excel que mostrava o funcionamento da gestão do programa no Assentamento PDS Osvaldo de Oliveira. Inicialmente, o foco era entender como as informações das chamadas públicas eram organizadas, quais atividades eram realizadas durante a execução dos editais e quais dificuldades estavam presentes no processo de acompanhamento. Observou-se que o gerenciamento do edital envolve diferentes informações, como quais famílias agricultoras estão participando, quais produtos foram selecionados, quantas entregas foram realizadas, quais destinos receberão os alimentos e quais são os valores a receber por agricultor, além das informações para o gerenciamento do edital, era necessário gerar documentos que comprovem as entregas.

5.1.1 Casos de uso do sistema

Os casos de uso do sistema foram definidos a partir das principais ações realizadas durante a gestão de uma chamada pública do PAA e para que ficasse coerente com o levantamento de requisitos. Como objetivo do sistema é organizar informações sobre editais, famílias, produtos, ciclos, destinos e entregas, foi necessário identificar quais pessoas interagem com a plataforma e quais atividades cada uma delas precisa realizar, que se resume em três atores principais, o administrador, o coordenador e o agricultor.

O administrador é o usuário que tem máximo acesso ao sistema, podendo criar coletivos, produtos, editais, aprovar usuários e modificar informações livremente no sistema. O segundo ator, o coordenador, é responsável por acompanhar a execução do edital, aprovar usuários do mesmo coletivo, cadastrar famílias, organizar produtos, criar editais, criar ciclos de entrega, registrar informações e acompanhar o andamento da chamada pública. Por fim, temos o agricultor que está relacionado ao acompanhamento das informações referentes à sua participação, como produtos previstos, entregas realizadas e valores a receber.

O primeiro caso de uso é o cadastro de usuários. Por meio dessa funcionalidade, uma pessoa interessada em acessar o sistema informa seus dados e solicita a criação de uma conta. Após esse cadastro, o usuário não recebe acesso imediato às funcionalidades internas, pois sua entrada depende da aprovação do coordena-

dor/administrador.

O segundo caso de uso é a aprovação de usuários. Essa funcionalidade permite que o coordenador visualize os usuários cadastrados do mesmo coletivo e aprove ou rejeite o acesso à plataforma. Essa etapa é crucial para manter a segurança do sistema e garantir que apenas pessoas vinculadas ao processo de participação no PAA possam acessar as informações.

Outro caso de uso central é o cadastro e gerenciamento de famílias. Essa funcionalidade permite registrar as famílias agricultoras no sistema, editar suas informações e associar usuários a uma determinada família. Esse vínculo é necessário para que o sistema consiga relacionar as entregas, os produtos e os valores a cada unidade produtiva.

O sistema também possui o caso de uso relacionado à criação e gerenciamento de editais. O edital representa a chamada pública do PAA e reúne as principais informações do processo. A partir dele, são organizados os produtos, os destinos, os ciclos e as entregas. O cadastro de produtos no sistema também é um dos principais casos de uso. Nele, o administrador registra os produtos que fazem parte do sistema e que são utilizados para a vinculação a um edital. O cadastro de destinos também compõe os principais casos de uso. Essa funcionalidade permite registrar as organizações e equipamentos de segurança alimentar que receberão os alimentos. A criação de ciclos de entrega é outro caso de uso fundamental. Como as entregas do PAA ocorrem de forma periódica, geralmente semanal, o sistema precisa organizar o edital em ciclos. Cada ciclo representa uma etapa de entrega e permite registrar os produtos movimentados em determinado período.

O registro das entregas realizadas representa uma das funcionalidades mais importantes do sistema. Por meio desse caso de uso, o coordenador informa quais famílias entregaram produtos, quais produtos foram entregues, em quais quantidades e para quais destinos. A partir desses dados, o sistema pode consolidar informações sobre o andamento do edital e calcular os valores correspondentes.

Também foi definido o caso de uso de acompanhamento do edital por meio de um dashboard. Esse dashboard tem como objetivo apresentar de forma visual e organizada o andamento da chamada pública, indicando informações como produtos entregues, valores acumulados, famílias participantes e ciclos já realizados.

A Tabela 5.1 apresenta uma síntese dos casos de uso identificados para o sistema.

Tabela 5.1: Principais casos de uso do sistema

Ator	Caso de uso	Descrição
Agricultor	Solicitar cadastro	Permite que uma pessoa solicite acesso ao sistema.
Coordenador/ Administrador	Aprovar usuário	Permite aprovar ou rejeitar usuários cadastrados na plataforma.
Coordenador/ Administrador	Gerenciar famílias	Permite cadastrar, editar e organizar as famílias agricultoras participantes.
Coordenador/ Administrador	Criar edital	Permite cadastrar uma chamada pública do PAA e suas informações principais.
Administrador	Gerenciar produtos	Permite cadastrar os produtos previstos no edital.
Administrador	Gerenciar destinos	Permite cadastrar as organizações e equipamentos que receberão os alimentos.
Coordenador/ Administrador	Criar ciclos de entrega	Permite organizar o edital em períodos de entrega.
Coordenador/ Administrador	Registrar entrega	Permite registrar os produtos entregues pelas famílias em cada ciclo.
Coordenador/ Administrador	Acompanhar dashboard	Permite visualizar o andamento do edital por meio de indicadores.
Coordenador/ Administrador	Gerar relatórios	Permite gerar documentos com informações sobre entregas, produtos e valores.
Agricultor	Consultar informações da participação	Permite acompanhar produtos entregues, quantidades registradas e valores a receber.
Administrador	Manter o sistema	Permite realizar ajustes, correções e evolução da plataforma.

5.2 Modelagem do domínio e estrutura dos dados

A modelagem do domínio foi construída a fim de atender as demandas necessárias para o acompanhamento do edital. O edital foi definido como uma das entidades centrais do sistema, pois representa a chamada pública que será gerenciada. Diretamente associado ao edital nós temos o plano original, entidade que contém os produtos previstos, os destinos das entregas e as famílias participantes.

O plano original é uma entidade que formaliza o início do edital, com informações essenciais como quais famílias irão participar, quais produtos e quais serão os destinos dos alimentos.

As famílias representam um grupo de usuários agricultores que participam do PAA. Dentro dos usuários pertencentes a família temos a entidade representante para sinalizar qual dos usuários será o responsável pelos registros nos relatórios gerados. Essa relação é importante porque as entregas e os valores a receber precisam ser vinculados corretamente ao responsável da família pela produção.

A entidade produto representa cada alimento que faz parte da chamada pública. No sistema, os produtos podem ser cadastrados independentemente como valores globais no sistema e podem ser associados ao edital com informações como valor unitário e quantidade prevista. Essa associação permite diferenciar os produtos dos diversos editais, controlar o que foi planejado e comparar posteriormente com o que foi efetivamente entregue.

Os destinos representam as organizações, instituições ou equipamentos de segurança alimentar que recebem os alimentos. O registro dos destinos permite acompanhar para onde os produtos estão sendo encaminhados e quanto cada destino irá receber.

Os ciclos de entrega foram modelados para representar a periodicidade das entregas realizadas durante a execução do edital. Como as chamadas públicas podem envolver entregas semanais ao longo de vários meses, a divisão em ciclos permite organizar melhor o acompanhamento do processo. Cada ciclo corresponde a um período específico e pode reunir diferentes entregas, produtos, famílias e destinos.

A entrega representa o registro efetivo dos produtos fornecidos em determinado ciclo. Por meio dela, o sistema armazena quais produtos foram entregues, em quais quantidades, por quais famílias e para quais destinos. Essa entidade é uma das mais importantes para o funcionamento do sistema, pois permite acompanhar a execução real do edital e gerar informações consolidadas para relatórios e conferência dos valores a receber.

Durante a modelagem, também foi identificada a necessidade de incluir o conceito de coletivo. Essa entidade permite representar agrupamentos internos de famílias ou agricultores, respeitando a forma de organização da comunidade. Com isso, o sistema não fica limitado a uma estrutura individualizada, podendo representar formas coletivas de participação e organização da produção.

Assim como a entidade de famílias, após o levantamento de requisitos, nasceu a

necessidade de inserir representantes nas entidades de destino e coletivos, pois além de serem necessárias as informações dos representantes em relatórios, o sistema registra uma relação formal de quem coordena o edital.

A Figura 5.1 apresenta uma proposta de modelo de domínio simplificado do sistema, destacando as principais entidades e suas relações.

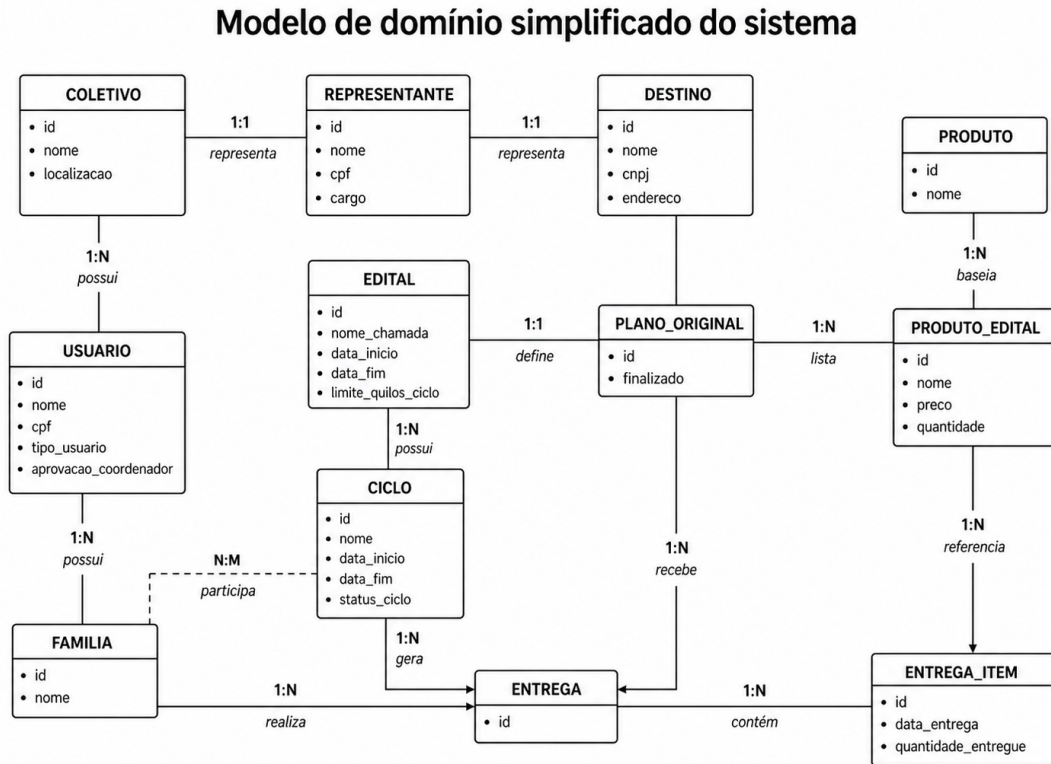


Figura 5.1: Modelo de domínio simplificado do sistema

5.3 Arquitetura geral do sistema

A arquitetura geral do sistema foi pensada para organizar o desenvolvimento em partes bem definidas, separando a interface utilizada pelos usuários das regras de negócio e do armazenamento das informações. Como o sistema precisa lidar com módulos e cada um com sua respectiva função, a aplicação foi estruturada para permitir que cada parte evolua de forma mais organizada, sem que uma alteração em uma funcionalidade comprometa todo o sistema.

O sistema foi desenvolvido como uma aplicação web, pois esse formato facilita o

acesso por diferentes dispositivos, como computadores, notebooks e celulares.

Um ponto considerado na arquitetura foi a necessidade de manter o sistema preparado para novas funcionalidades, a arquitetura foi pensada de forma modular, permitindo que novos módulos sejam incorporados progressivamente, sem afetar o comportamento atual do sistema.

A estrutura geral é composta por duas partes principais: o frontend e o backend. O frontend é responsável pela interface com o usuário, apresentando as telas, formulários, tabelas e informações necessárias para a gestão do PAA. O backend é responsável por receber as requisições da interface, aplicar as regras de negócio e realizar a comunicação com o banco de dados.

A intenção de manter o frontend e o backend separados era de dar independência ao desenvolvimento de cada um. Mesmo que um complete o outro, funcionalidades como filtros podem ser aplicadas ao backend sem que seja necessário derrubar toda a aplicação.

5.4 Implementação do backend

Seguindo princípios de arquitetura limpa, a estrutura de pastas do backend foi dividida em application, domain, infrastructure e web. A application é responsável pelas interfaces dos services que serão implementados. O domain representa todas as entidades do sistema que foram explicadas em seções anteriores, assim como DTOs e as interfaces que são implementadas em tempo de execução para o armazenamento no banco de dados. A infrastructure é responsável por armazenar configurações necessárias, a implementação dos services que dão a alma para o sistema e funções utilitárias para o sistema. A pasta web é responsável por armazenar os controllers que são o ponto de entrada das requisições, sejam elas vindas do frontend ou de alguma ferramenta que realiza requisições HTTP. Na Figura 5.2 temos a ilustração da organização de diretórios do projeto.

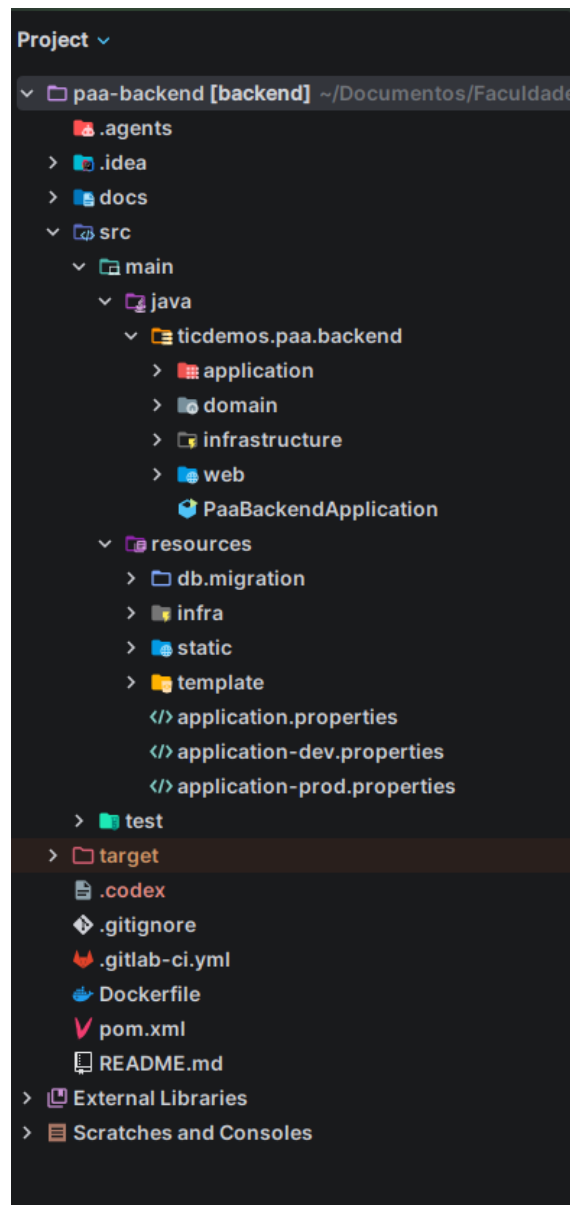


Figura 5.2: Estrutura de diretórios do backend

A tecnologia utilizada para implementar o backend foi o Java 17, juntamente com o framework Spring Boot 3. Essa escolha permitiu estruturar a aplicação de forma modular, facilitando a criação de APIs, a organização das regras de negócio e a integração com o banco de dados. Para a criação dos endpoints e dos controllers responsáveis por receber as requisições da interface, foi utilizado o Spring Web, que permitiu organizar as rotas HTTP utilizadas pelo frontend.

Para melhorar a experiência de desenvolvimento e simplificar a comunicação com o banco de dados, foi utilizado o Spring Data JPA em conjunto com o Hibernate, uma ferramenta de mapeamento objeto-relacional (ORM). Com o uso dessas tecnologias,

as entidades do sistema podem ser representadas por classes Java, permitindo que operações de armazenamento e consulta sejam realizadas por meio de métodos e repositórios da aplicação. Dessa forma, reduz-se a necessidade de escrever comandos SQL diretamente.

O banco de dados escolhido foi o PostgreSQL, por ser uma ferramenta open-source, robusta e amplamente utilizada em aplicações web. Além disso, o PostgreSQL oferece bom desempenho tanto para sistemas em estágio inicial, como MVPs, quanto para aplicações maiores, que exigem maior consistência, confiabilidade e capacidade de expansão. Durante o desenvolvimento, o pgAdmin foi utilizado como ferramenta de apoio para visualizar tabelas, acompanhar registros e verificar informações armazenadas no banco de dados.

Para lidar com questões de segurança da aplicação, foi inserido no projeto o Spring Security, possibilitando o controle de acesso às rotas restritas e o bloqueio de endpoints para usuários não autenticados. Além disso, foi adotado o uso de JWT, permitindo que as requisições ao backend fossem realizadas de forma autenticada e associadas ao usuário logado. Com isso, tornou-se possível controlar o acesso às funcionalidades do sistema de acordo com o perfil de cada usuário.

Para os testes unitários, foram utilizadas as bibliotecas JUnit e Mockito. O JUnit permite estruturar e executar os testes automatizados, enquanto o Mockito possibilita simular comportamentos de componentes da aplicação, como serviços e repositórios. Essas ferramentas contribuem para validar processos críticos, como criação, atualização e consulta de dados, além de ajudar a garantir que mudanças futuras no código não comprometam comportamentos já implementados. O gerenciamento das dependências, a execução dos testes e a geração do pacote da aplicação foram organizados por meio do Maven, mantendo uma estrutura padronizada para o projeto backend.

Na documentação da API, foi utilizado o OpenAPI/Swagger, por ser uma ferramenta open-source que gera uma interface interativa para visualização e teste dos endpoints da aplicação. Com isso, torna-se possível consultar os recursos disponíveis, verificar os parâmetros esperados e testar requisições diretamente pela própria aplicação, sem a necessidade inicial de ferramentas externas.

5.4.1 Desafios de implementação do backend

Com o objetivo de apresentar os principais desafios técnicos enfrentados durante o desenvolvimento do backend, esta seção analisa implementações que são complexas na modelagem, integração e controle das entidades do sistema.

O primeiro trecho de código é referente à implementação responsável pelo cadastro de produtos no plano original, destacando seu funcionamento e as decisões adotadas para manter a consistência dos dados.

Código: 5.1: Serviço de adicionar produtos ao edital

```
1 @Override
2 public List<ProdutoEditalDTO> adicionarProdutoEdital(List<
    Long> idProdutos, Long idPlanoOriginal) {
3
4     PlanoOriginal planoOriginal = planoOriginalRepository.
        findById(idPlanoOriginal)
5         .orElseThrow(() -> new RuntimeException("Erro ao
            encontrar o plano original"));
6
7     List<ProdutoEdital> produtosEdital = planoOriginal.
        getProdutos();
8
9     idProdutos.forEach(id -> {
10         Optional<Produto> oProduto = this.produtoRepository.
            findById(id);
11         if (oProduto.isPresent()) {
12             Produto produto = oProduto.get();
13
14             ProdutoEdital produtoEdital = new ProdutoEdital();
15             produtoEdital.setNome(produto.getNome());
16             produtoEdital.setPreco(new BigDecimal("0.0"));
17             produtoEdital.setQuantidade(0L);
18             produtoEdital.setPlanoOriginal(planoOriginal);
19             produtosEdital.add(produtoEditalRepository.save(
                produtoEdital));
```

```

20     }
21 });
22
23 planoOriginal.setProdutos(produtosEdital);
24
25 return planoOriginalRepository.save(planoOriginal).
    getProdutos()
26     .stream()
27     .filter(produtoEdital ->
28         Objects.isNull(produtoEdital.
                getDestinoProdutoEdital()) && Objects.
                isNull(produtoEdital.
                getFamiliaProdutoEdital())
29     )
30     .map(PlanoOriginalService::toProdutoEditalDTO)
31     .toList();
32 }

```

Essa função é responsável por adicionar produtos a um plano original. O método recebe como parâmetros uma lista com os identificadores dos produtos previamente cadastrados no sistema e o identificador do plano original que será atualizado. Inicialmente, o plano é consultado no banco de dados por meio do repositório correspondente. Caso o registro não seja encontrado, uma exceção é lançada, impedindo a continuidade da operação com dados inexistentes.

Após a recuperação do plano original, é obtida a lista de produtos já associados a ele. Em seguida, o código percorre os identificadores recebidos. Para cada item, é realizada uma consulta no repositório de produtos. Quando o produto é localizado, uma nova entidade `ProdutoEdital` é criada e preenchida com as informações necessárias. Essa entidade representa a relação entre um produto cadastrado globalmente na aplicação e sua utilização dentro de um edital específico.

Outro desafio de implementação foi a criação de ciclos dentro do sistema. O código abaixo mostra a implementação da criação de um ciclo.

Código: 5.2: Serviço para criação de ciclos dentro do sistema

```

1 @Override
2 public CicloDTO createCiclo(CicloRequest cicloDTO, Long
   idEdital) {
3
4     Edital edital = editalService.getEdital(idEdital);
5
6     Ciclo ciclo = new Ciclo();
7     ciclo.setEntregas(new ArrayList<>());
8
9     Set<Familia> familias = edital.getPlanoOriginal().
       getFamiliasProdutoEdital()
10         .stream()
11         .map(FamiliaProdutoEdital::getFamilia)
12         .collect(Collectors.toSet());
13
14     edital.getPlanoOriginal()
15         .getFamiliasProdutoEdital()
16         .forEach(familiaProdutoEdital -> {
17             Entrega entrega = new Entrega();
18             entrega.setCiclo(ciclo);
19             entrega.setFamilia(familiaProdutoEdital.
               getFamilia());
20             entrega.setItens(new ArrayList<>());
21             entrega.setDestino(null);
22
23             familiaProdutoEdital.getProdutos()
24                 .forEach(produtoEdital -> {
25                     EntregaItem entregaItem = new
                       EntregaItem();
26
27                     entregaItem.setEntrega(entrega);
28                     entregaItem.setProdutoEdital(
                       produtoEdital);
29                     entregaItem.setQuantidadeEntregue(0);

```

```

30         entrega.addItem(entregaItem);
31
32     });
33
34     ciclo.addEntregas(entrega);
35
36     });
37
38     edital
39     .getPlanoOriginal()
40     .getDestinosProdutoEdital()
41     .forEach(destinoProdutoEdital -> {
42         Entrega entrega = new Entrega();
43         entrega.setCiclo(ciclo);
44         entrega.setFamilia(null);
45         entrega.setItens(new ArrayList<>());
46         entrega.setDestino(destinoProdutoEdital.
47             getDestino());
48
49         destinoProdutoEdital.getProdutos()
50         .forEach(produtoEdital -> {
51             EntregaItem entregaItem = new EntregaItem
52                 ();
53
54             entregaItem.setEntrega(entrega);
55             entregaItem.setProdutoEdital(
56                 produtoEdital);
57             entregaItem.setQuantidadeEntregue(0);
58             entrega.addItem(entregaItem);
59
60         });
61
62     ciclo.addEntregas(entrega);

```

```

61         });
62
63         StatusCiclo statusCiclo = isBetween(LocalDate.now(),
64             cicloDTO.dataInicio(), cicloDTO.dataFim())
65             ? StatusCiclo.ABERTO
66             : StatusCiclo.AGUARDANDO;
67
68         ciclo.setNome(cicloDTO.nome());
69
70         verifyDateCiclo(cicloDTO, edital);
71         ciclo.setDataInicio(cicloDTO.dataInicio());
72         ciclo.setDataFim(cicloDTO.dataFim());
73
74         ciclo.setAprovacaoCoordenacao(false);
75
76         ciclo.setStatusCiclo(statusCiclo);
77
78         ciclo.setEdital(edital);
79         ciclo.setFamilias(familias);
80
81         edital.addCiclo(ciclo);
82
83         try {
84             Ciclo cicloSaved = cicloRepository.save(ciclo);
85             return CicloService.toDto(cicloSaved);
86         } catch (Exception e) {
87             log.info("Erro ao salvar o ciclo: {}", e.getMessage());
88             ;
89         }
90
91         return null;
92     }

```

Nesse trecho é apresentada a função responsável pela criação de um ciclo de entregas associado a um edital. O método recebe como parâmetros um objeto

CicloRequest, contendo os dados necessários para o cadastro, e o identificador do edital ao qual o ciclo será vinculado. Inicialmente, o edital é recuperado no banco de dados e, em seguida, uma nova entidade Ciclo é criada com sua lista de entregas inicializada.

Após essa etapa, são identificadas as famílias participantes a partir das associações previamente cadastradas no plano original do edital. Para cada relação entre família e produto, é criada uma entrega vinculada ao ciclo e à respectiva família. Em seguida, são gerados os itens da entrega, relacionando cada produto previsto e inicializando sua quantidade entregue com o valor zero. Esse processo permite que o ciclo seja criado já contendo a estrutura necessária para o registro das entregas realizadas posteriormente.

O mesmo procedimento é aplicado aos destinos cadastrados no plano original. Para cada destino, é criada uma nova entrega sem associação direta com uma família, mas vinculada aos produtos previstos para aquele local. Dessa forma, o ciclo reúne tanto as entregas organizadas por família quanto aquelas relacionadas aos destinos definidos no edital.

Outro ponto tratado pelo método é a definição automática do status inicial do ciclo. Caso a data atual esteja dentro do período informado, o ciclo é criado com o status ABERTO, caso contrário, permanece como AGUARDANDO. Antes do armazenamento, as datas são validadas, e também são definidos o nome, a situação de aprovação da coordenação, o edital relacionado e o conjunto de famílias participantes.

Também podemos destacar a geração automática dos termos de recebimento. Essa funcionalidade utiliza os dados cadastrados durante a execução do ciclo para preencher um documento padronizado, reduzindo a necessidade de elaboração manual. No trecho de código apresentado abaixo, é possível observar a implementação responsável por gerar esse documento.

Código: 5.3: Serviço geração do termo de recebimento

```
1  
2 public byte[] gerarTermoRecebimento(  
3     Long cicloId,  
4     UUID userUuid,
```

```

5         Long destinoUuid
6    ) throws IOException, Docx4JException, JAXBException {
7
8        Ciclo ciclo = cicloService.getCiclo(cicloId);
9
10       Usuario usuario = usuarioService.getUsuario(userUuid);
11       Coletivo coletivo = usuario.getColetivo();
12
13       Destino destino = destinoService.getDestino(destinoUuid);
14
15       WordprocessingMLPackage pkg =
16           WordprocessingMLPackage.load(
17               new ClassPathResource("template/
18                   TermoRecebimentoTemplate.docx").
19                   getInputStream()
20           );
21
22       List<Item> itens = new ArrayList<>();
23       ciclo.getEntregas()
24           .stream()
25           .filter(entrega -> Objects.nonNull(entrega.getDestino(
26               )))
27           .filter(entrega -> entrega.getDestino().getId().
28               equals(destino.getId()))
29           .forEach(entrega -> {
30               entrega.getItems().forEach(entregaItem -> {
31                   Item item = new Item(
32                       entregaItem.getProdutoEdital().
33                           getNome(),
34                       "Kg",
35                       entregaItem.getQuantidadeEntregue(),
36                       entregaItem.getProdutoEdital().
37                           getPreco().doubleValue()
38                   );

```



```

33         itens.add(item);
34     });
35 });
36
37 List<Object> tables = pkg.getMainDocumentPart().
    getJAXBNodesViaXPath("//w:tbl", true);
38 if (tables.isEmpty()) {
39     throw new RuntimeException("Nenhuma tabela encontrada
        no documento.");
40 }
41
42 Tbl table = unwrap(tables.get(0));
43
44 List<Tr> rows = table.getContent().stream()
45     .filter(o -> o instanceof Tr)
46     .map(o -> (Tr) o)
47     .collect(Collectors.toList());
48
49 if (rows.size() < 3) {
50     throw new RuntimeException("Tabela precisa ter pelo
        menos: header, linha modelo e total.");
51 }
52
53 Tr modelRow = rows.get(4);
54
55 Tr totalRow = rows.get(rows.size() - 1);
56
57 table.getContent().remove(modelRow);
58
59 double totalGeral = 0.0;
60
61 for (Item item : itens) {
62     Tr newRow = XmlUtils.deepCopy(modelRow);
63

```

```

64         double totalItem = item.quantidade * item.
           valorUnitario;
65         totalGeral += totalItem;
66
67         setCellText(newRow, 0, item.produto);
68         setCellText(newRow, 1, item.unidade);
69         setCellText(newRow, 2, formatNumber(item.quantidade))
           ;
70         setCellText(newRow, 3, formatNumber(item.
           valorUnitario));
71         setCellText(newRow, 4, formatNumber(totalItem));
72
73         table.getContent().add(table.getContent().size() - 1,
           newRow);
74     }
75
76     setCellText(totalRow, 1, formatNumber(totalGeral));
77
78     Map<String, String> finalPage = new HashMap<>();
79
80     finalPage.put("destino", destino.getNome());
81     finalPage.put("cnpj", Utils.formatCnpj(destino.getCnpj())
       );
82
83     finalPage.put("endereco", destino.getEndereco());
84     finalPage.put("municipio", "Rio de Janeiro/RJ");
85     finalPage.put("cep", "21.450-250");
86     finalPage.put("representanteDestino", destino.
       getRepresentante().getNome());
87     finalPage.put("cpfRepresentante", Utils.formatCpf(destino
       .getRepresentante().getCpf()));
88     finalPage.put("dataRecebimento", LocalDate.now().format(
       DateTimeFormatter.ofPattern("dd/MM/yyyy")));
89

```

```

90     finalPage.put("assentamento", coletivo.getNome());
91
92     finalPage.put("valorTotalFinal", formatNumber(totalGeral)
93         );
94     finalPage.put("municipioFinal", "MACAE/RJ");
95     finalPage.put("dataFinal", LocalDate.now().format(
96         DateTimeFormatter.ofPattern("dd/MM/yyyy")));
97     finalPage.put("representanteRecebedora", destino.
98         getRepresentante().getNome());
99     finalPage.put("cpfRepresentanteRecebedora", Utils.
100         formatCpf(destino.getRepresentante().getCpf()));
101
102     finalPage.put("representanteRornecedora", coletivo.
103         getRepresentante().getNome());
104     finalPage.put("cargoRepresetanteFornecedora", coletivo.
105         getRepresentante().getCargo());
106     finalPage.put("cpfRepresentanteFornecedora", Utils.
107         formatCpf(coletivo.getRepresentante().getCpf()));
108
109     pkg.getMainDocumentPart().variableReplace(finalPage);
110
111     ByteArrayOutputStream baos = new ByteArrayOutputStream();
112     pkg.save(baos);
113
114     return baos.toByteArray();
115 }

```

A função recebe como parâmetros o identificador do ciclo, o identificador do usuário responsável pela solicitação e o identificador do destino relacionado ao recebimento. As informações são utilizadas para recuperar o ciclo, o usuário e o destino no banco de dados. A partir do usuário, também é obtido o coletivo ao qual ele pertence, uma vez que seus dados são necessários para identificar a organização fornecedora no termo.

Em seguida, o método carrega um arquivo no formato DOCX que funciona como

modelo para o documento. Esse arquivo possui uma estrutura previamente definida, com campos de texto e uma tabela que serão preenchidos durante a execução, o template criado está armazenado no sistema. Dessa forma, a aplicação consegue manter a padronização visual do termo de recebimento sem precisar construir toda a formatação do documento diretamente no código.

Após o carregamento do modelo, são selecionadas as entregas pertencentes ao ciclo que estão associadas ao destino informado. Para cada entrega encontrada, o método percorre seus itens e reúne informações como nome do produto, unidade de medida, quantidade entregue e valor unitário. Esses dados são organizados em uma lista auxiliar, que posteriormente será utilizada para preencher a tabela existente no documento.

Na etapa seguinte, o método localiza a tabela no arquivo DOCX e identifica a linha utilizada como modelo para os produtos. Essa linha é copiada para cada item da lista, preservando a formatação previamente definida no documento. Em cada nova linha são inseridos o nome do produto, a unidade de medida, a quantidade entregue, o valor unitário e o valor total do item, calculado pela multiplicação entre a quantidade e o preço unitário. Durante esse processo, os valores individuais também são acumulados para obtenção do valor total geral do termo.

Além da tabela de produtos, o método preenche os demais campos do documento por meio de um mapa de variáveis. Entre as informações adicionadas estão o nome e o CNPJ do destino, o endereço, os dados dos representantes, o nome do coletivo, a data de recebimento e o valor total calculado. Os números e documentos pessoais são formatados antes de serem inseridos, garantindo que valores monetários, CPF e CNPJ sejam apresentados de maneira adequada no arquivo final.

Após o preenchimento de todas as informações, o documento é armazenado temporariamente em memória por meio de um fluxo de bytes. Ao final, o método retorna um vetor de bytes, permitindo que o termo seja disponibilizado para download pela camada responsável pela requisição, sem a necessidade de salvar uma cópia permanente do documento no servidor.

Por último um processo que é complexo em sua estrutura e possui um comportamento fundamental na segurança da aplicação. O processo de login, abaixo, segue o trecho de código do service que valida o login da requisição.

Código: 5.4: Serviço de login

```
1 @Override
2 public void login(AuthenticationRequest authenticationRequest
3     , HttpServletResponse response) {
4
5     if(!this.userService.usuarioAprovado(
6         authenticationRequest.getUsername())){
7
8         throw new UsuarioNaoLiberadoException("Usuario nao
9             liberado");
10    }
11
12    authenticationManager.authenticate(
13        new UsernamePasswordAuthenticationToken(
14            authenticationRequest.getUsername(),
15            authenticationRequest.getPassword()));
16
17    CustomUserDetail userDetails = userService
18        .loadUserByUsername(authenticationRequest.
19            getUsername());
20
21    String jwtToken = jwtUtil.generateToken(userDetails);
22    response.addCookie(jwtUtil.createJwtCookie(COOKIE_NAME,
23        jwtToken));
24
25    processRefreshToken(response, userDetails.getUuid());
26 }
```

A função recebe um objeto `AuthenticationRequest`, contendo o nome de usuário e a senha informados, além do objeto `HttpServletResponse`, utilizado para adicionar os dados de autenticação à resposta HTTP. Inicialmente, o método verifica se o usuário está aprovado para acessar o sistema. Caso o cadastro ainda não tenha sido liberado, é lançada a exceção `UsuarioNaoLiberadoException`, interrompendo o processo antes mesmo da validação das credenciais.

Após a verificação da aprovação, o nome de usuário e a senha são encapsulados em um objeto `UsernamePasswordAuthenticationToken`. Esse objeto é enviado ao `AuthenticationManager`, responsável por validar as credenciais de acordo com as configurações de segurança da aplicação. Caso os dados estejam incorretos, o processo de autenticação é interrompido e o acesso ao sistema é negado. Com a autenticação realizada, os dados completos do usuário são recuperados por meio do `UserDetailsService`. Essas informações são utilizadas para gerar um token JWT, que identifica o usuário autenticado nas requisições posteriores. Em seguida, o token é armazenado em um cookie e adicionado à resposta HTTP, permitindo que o navegador envie automaticamente nas próximas comunicações com o backend.

Além do token de acesso, o método chama a função responsável pelo processamento do refresh token. Para isso, utiliza o identificador único do usuário autenticado. Esse segundo token permite renovar a autenticação quando o JWT principal expirar, evitando que o usuário precise informar novamente suas credenciais durante uma sessão válida.

5.5 Implementação do frontend

O frontend foi pensado para tornar o sistema intuitivo e facilitar o uso pelos diferentes perfis de usuários. No caso do coordenador e do administrador, a interface busca auxiliar na administração dos editais do PAA. Já para o agricultor, o objetivo é permitir o acompanhamento claro do andamento de sua participação no edital.

No desenvolvimento das telas e dos componentes do sistema, foi utilizada a biblioteca React 19. Para personalizar a interface e aproximar o projeto dos padrões visuais presentes em aplicações web atuais, foi escolhida a biblioteca MUI. Além de ser open-source, essa biblioteca implementa os princípios do Material Design, proposto pelo Google, contribuindo para a construção de uma interface padronizada, responsiva e de fácil utilização. A estilização dos componentes também utiliza o Emotion, tecnologia associada ao ecossistema do MUI e utilizada para organizar estilos aplicados à interface.

O TypeScript está presente em todo o desenvolvimento do frontend, pois contribui para manter maior coerência entre os dados manipulados pela aplicação. Como

o sistema realiza troca constante de informações com o backend, os dados recebidos e enviados são tipados, o que reduz erros durante o desenvolvimento e facilita a manutenção do código.

Além da biblioteca de design, o frontend utiliza outras ferramentas importantes para a organização da aplicação. O roteamento das páginas é realizado com o React Router DOM, permitindo controlar a navegação entre as diferentes telas do sistema. Para o gerenciamento de estado, são utilizados o Redux Toolkit e o React Redux, que centralizam o estado da aplicação em uma store e organizam as alterações por meio de actions e reducers. Essa abordagem contribui para tornar o comportamento da interface mais previsível, principalmente em telas que dependem de dados compartilhados entre diferentes componentes.

A comunicação com o backend é feita por meio da biblioteca Axios, responsável por realizar requisições HTTP para a API do sistema. Também são utilizadas bibliotecas auxiliares para formatação e processamento de datas, contribuindo para a apresentação adequada das informações ao usuário. O Vite foi utilizado como ferramenta de desenvolvimento e construção do frontend, facilitando a execução local da aplicação e a geração dos arquivos de produção. Além disso, o ESLint foi utilizado para auxiliar na padronização do código e na identificação de problemas durante o desenvolvimento. Todas as bibliotecas utilizadas no frontend são gerenciadas pelo arquivo package.json, por meio do npm, que organiza as dependências necessárias para o funcionamento e a manutenção da aplicação.

5.5.1 Telas implementadas

Abaixo serão apresentadas as telas do sistema, juntamente com as descrições de cada uma. Na figura 5.3 temos a primeira tela, a de login, onde os usuários já cadastrados podem inserir suas credenciais e realizar o acesso e os interessados no sistema podem requisitar o acesso através do botão de criar conta.

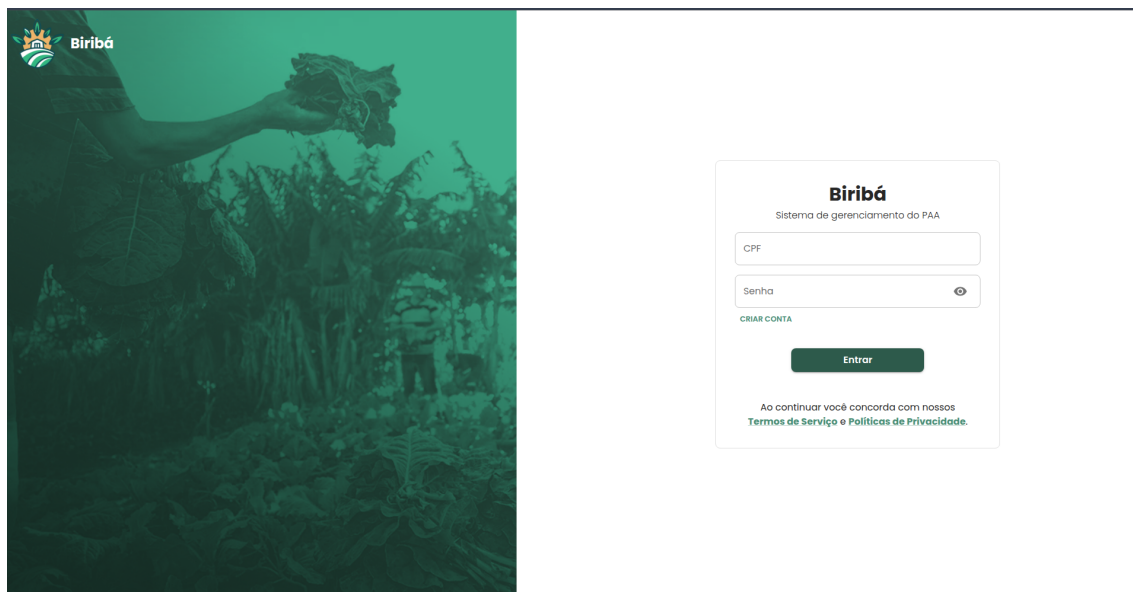


Figura 5.3: Login do sistema

Após o login o usuário segue para a seleção do edital que, no caso do coordenador, deseja verificar o andamento ou realizar o gerenciamento. Na tela 5.4 temos todos os editais disponíveis para a seleção. Vale ressaltar que o coordenador só pode acompanhar o andamento dos editais que são do seu coletivo. Um filtro é realizado no backend para que os dados que apareçam na interface sigam essa regra. O administrador do sistema pode ver todos os editais existentes, e o agricultor tem acesso semelhante ao do coordenador, podendo ver somente os editais dos quais participa. Nesta tela também podemos ver um botão de criar novos editais, permitindo que o coordenador ou o administrador inicie o gerenciamento de uma nova chamada pública.

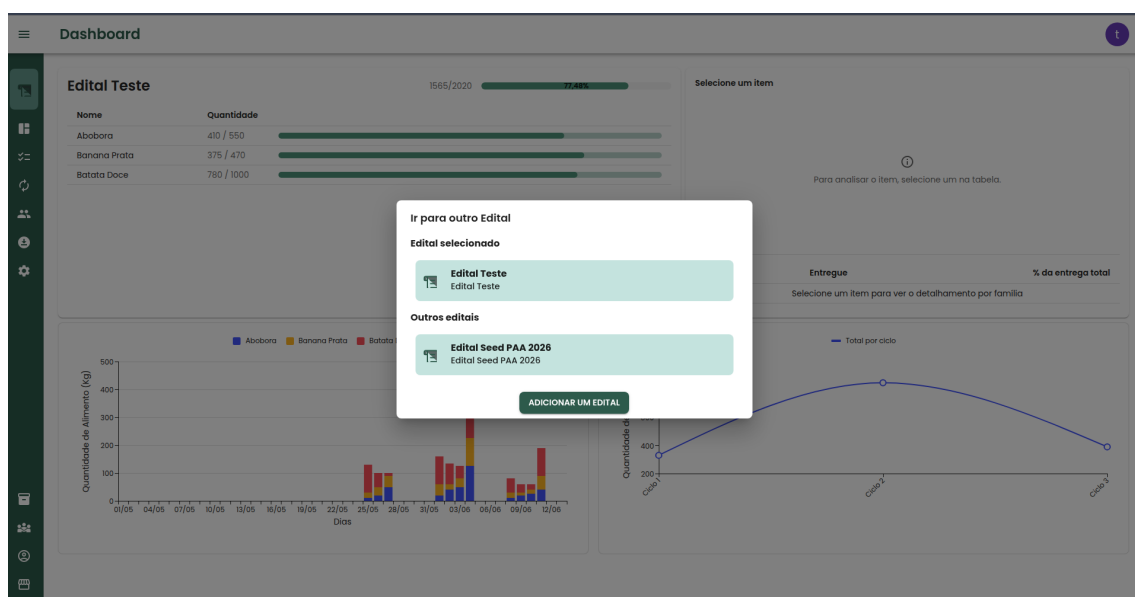


Figura 5.4: Seleção de editais

Em seguida, na criação do edital, o coordenador deverá preencher o plano original, definindo quais famílias participarão do edital, quais produtos serão ofertados e quais serão os destinos das entregas. A Figura 5.5 exemplifica um plano original parcialmente preenchido, contendo apenas as famílias e os produtos. Nessa mesma figura, observa-se a presença de botões para adicionar uma nova família ao edital e para incluir um novo produto.

Na Figura 5.6, é apresentada a aba de produtos por família já preenchida no plano original. A primeira coluna da tabela corresponde aos produtos que farão parte do edital selecionado, enquanto a segunda coluna indica o valor do quilo de cada produto. As demais colunas, com exceção da última, representam as famílias participantes do edital.

Em cada linha, as células associadas às famílias indicam a quantidade de determinado produto que cada família deverá entregar ao longo do edital. A última coluna apresenta o valor total, em reais, correspondente à quantidade total prevista para aquele produto. Já a última célula de cada coluna referente às famílias mostra o valor total que cada família deverá receber ao final do edital. Por fim, a célula localizada na última linha e na última coluna da tabela apresenta o valor total previsto para o edital, considerando todos os produtos e todas as famílias participantes.

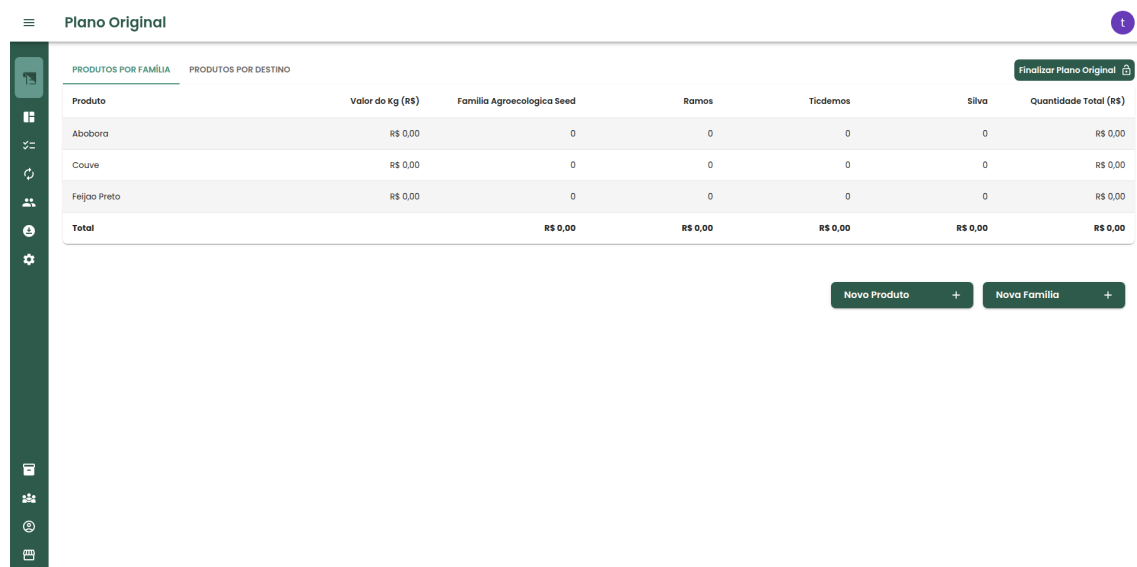
Na Figura 5.7, é apresentado um exemplo de preenchimento da aba de produtos

por destino. Essa tabela possui estrutura semelhante à aba anterior, mas, nesse caso, as colunas representam os destinos das entregas. Cada célula das colunas de destino indica a quantidade de determinado produto que será enviada para aquele destino ao longo do edital.

A última coluna dessa tabela é utilizada como referência para orientar o coordenador na distribuição dos produtos entre os destinos. Ela é preenchida a partir das quantidades previamente definidas para os agricultores na aba de produtos por família, permitindo verificar quanto de cada produto ainda precisa ser distribuído. Dessa forma, o coordenador consegue organizar a destinação dos alimentos de acordo com o total previsto no plano original.

Na página do plano original, além dos botões para adicionar produtos, famílias e destinos, há também o botão “Finalizar Plano Original”. Esse botão tem como objetivo encerrar a etapa de edição das tabelas e dar início ao andamento do edital. Após a finalização, as tabelas deixam de permitir alterações, garantindo que os dados definidos no plano original sejam preservados durante a execução do edital.

Observa-se que, nas Figuras 5.6 e 5.8, esse botão aparece desabilitado, e os botões de inclusão não estão presentes na tela. Isso indica que o plano original já foi finalizado e que a edição das informações foi bloqueada.



Plano Original

PRODUTOS POR FAMÍLIA PRODUTOS POR DESTINO

Finalizar Plano Original

Produto	Valor do Kg (R\$)	Família Agroecológica Seed	Ramos	Tiçdemos	Silva	Quantidade Total (R\$)
Abobora	R\$ 0,00	0	0	0	0	R\$ 0,00
Couve	R\$ 0,00	0	0	0	0	R\$ 0,00
Feijão Preto	R\$ 0,00	0	0	0	0	R\$ 0,00
Total		R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00

Novo Produto + Nova Família +

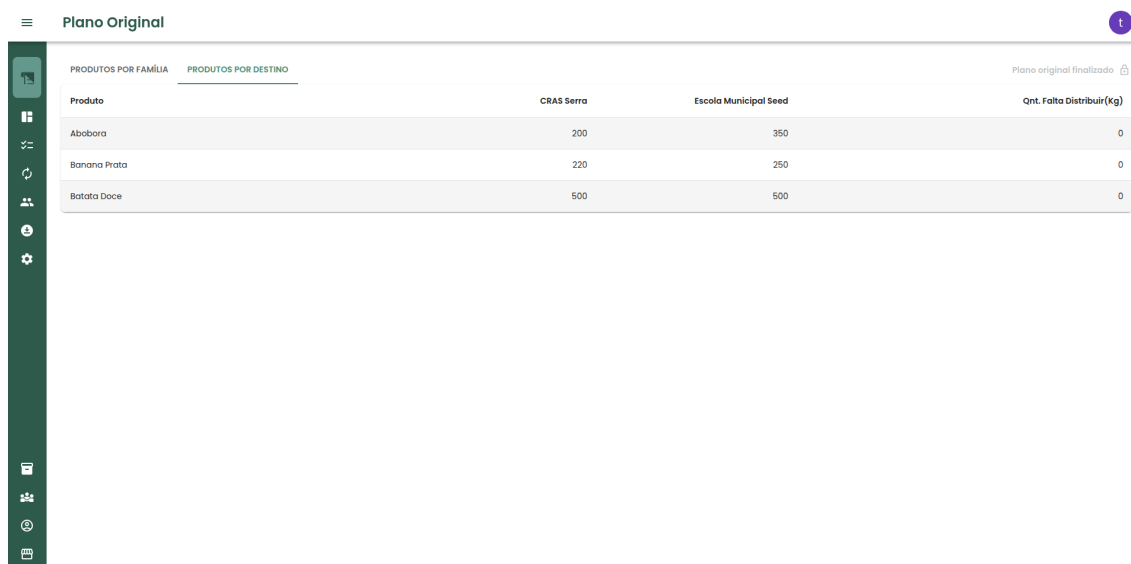
Figura 5.5: Plano original aba produto por família parcialmente preenchido

Plano Original						
PRODUTOS POR FAMÍLIA		PRODUTOS POR DESTINO				
Produto	Valor do Kg (R\$)	Ticdemos	Familia Agroecologica Seed	Silva	Ramos	Quantidade Total (R\$)
Abobora	R\$ 4,99	50	100	150	250	R\$ 2.744,50
Banana Prata	R\$ 2,65	100	70	100	200	R\$ 1.245,50
Batata Doce	R\$ 3,70	300	200	100	400	R\$ 3.700,00
Total		R\$ 1.624,50	R\$ 1.424,50	R\$ 1.383,50	R\$ 3.257,50	R\$ 7.690,00

Figura 5.6: Plano original aba produto por família completo

Plano Original			
PRODUTOS POR FAMÍLIA		PRODUTOS POR DESTINO	
Produto	CRAS Serra	Escola Municipal Seed	Qnt. Falta Distribuir(kg)
Abobora	0	0	0
Couve	0	0	0
Feijão Preto	0	0	0

Figura 5.7: Plano original aba produto por destino parcialmente preenchido



Produto	CRAS Serra	Escola Municipal Seed	Qnt. Falta Distribuir(kg)
Abacora	200	350	0
Banana Prata	220	250	0
Batata Doce	500	500	0

Figura 5.8: Plano original aba produto por destino completo

Após a finalização do plano original, inicia-se a etapa de acompanhamento do andamento do edital por meio da tela de ciclos de entrega. Nessa tela, o coordenador pode criar os ciclos, definindo suas datas de início e fim, além de registrar a quantidade de produtos entregue por cada família e a quantidade destinada a cada local de entrega.

A Figura 5.9 exemplifica os três status possíveis dos ciclos já criados. Nessa tela, observa-se também a existência de dois controles principais: um relacionado às famílias e outro aos destinos. No controle de famílias, o coordenador registra as entregas realizadas em cada ciclo, informando a quantidade de produtos entregue por cada família. Para auxiliar esse processo, a tabela apresenta uma coluna de apoio que indica a quantidade ainda faltante de cada produto, considerando os valores definidos previamente no plano original.

O controle de destinos possui funcionamento semelhante. Nele, o coordenador define a quantidade de produtos que será encaminhada para cada destino em determinado ciclo. Ao lado das informações de entrega, o sistema apresenta a quantidade total prevista para cada destino ao longo do edital, permitindo acompanhar se a distribuição está de acordo com o plano original. Além disso, ao lado do nome de cada destino, há um botão para baixar o relatório necessário à comprovação da entrega daquele ciclo.

Assim como ocorre na página do plano original, o ciclo de entrega pode ser finalizado antes da data prevista para o seu encerramento. Ao finalizar um ciclo, seu status é alterado e a edição das informações passa a ser bloqueada, garantindo maior controle sobre os dados registrados. No entanto, diferentemente do plano original, um ciclo finalizado pode ser reaberto, caso seja necessário corrigir ou complementar alguma informação posteriormente.

Ciclos de Entrega

Todos os Ciclos

- Ciclo 1**
24/05/2026 a 30/05/2026
Encerrado
Ver Detalhes
- Ciclo 2**
31/05/2026 a 06/06/2026
Aberto
Ver Detalhes
- Ciclo 3**
07/06/2026 a 13/06/2026
Aguardando
Ver Detalhes

Ciclo 2
31/05/2026 a 06/06/2026
Aberto
Finalizar Ciclo

Controle de Famílias

Produto	Data	Qtd. Entrega	Qtd. Faltante	Editar
Abobora	01/06/2026	20	20	
Banana Prata	01/06/2026	40	40	
Batata Doce	01/06/2026	100	100	

Controle de Local

Produto	CRAS Serra	Escola Municipal Seed	TOTAL
Abobora	80 / 80	155 / 155	0
Banana Prata	135 / 40	55 / 150	0
Batata Doce	210 / 210	210 / 210	0

Gerar Recibo

Figura 5.9: Ciclos e Entregas

Após a finalização do ciclo de entrega, o coordenador pode baixar os relatórios referentes àquele período para cada destino. A Figura 5.10 apresenta, por meio de cards, todos os destinos vinculados ao edital, juntamente com suas principais informações. Em cada card, é possível baixar o relatório de recebimento de alimentos, utilizado para comprovar as entregas realizadas naquele destino. Dessa forma, os documentos são gerados pelo coordenador, mas registram informações que servem como comprovação das entregas feitas pelas famílias agricultoras.

Além disso, nessa mesma tela, há o botão para baixar o relatório de pagamento. Esse relatório sintetiza as entregas realizadas por família, apresentando a quantidade entregue e o valor que cada uma deverá receber ao final do ciclo.

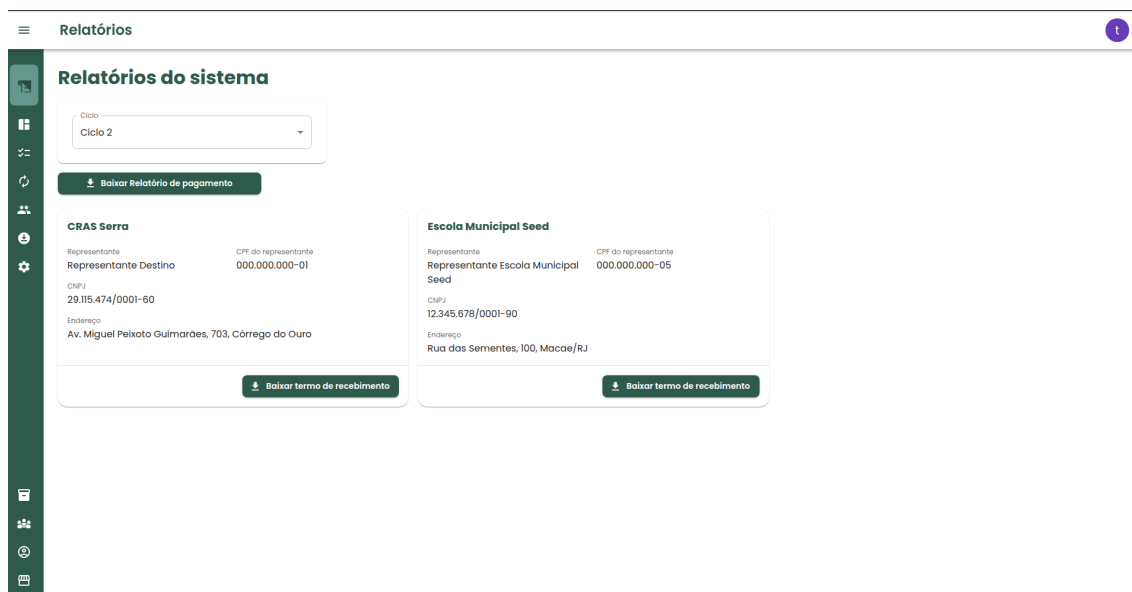
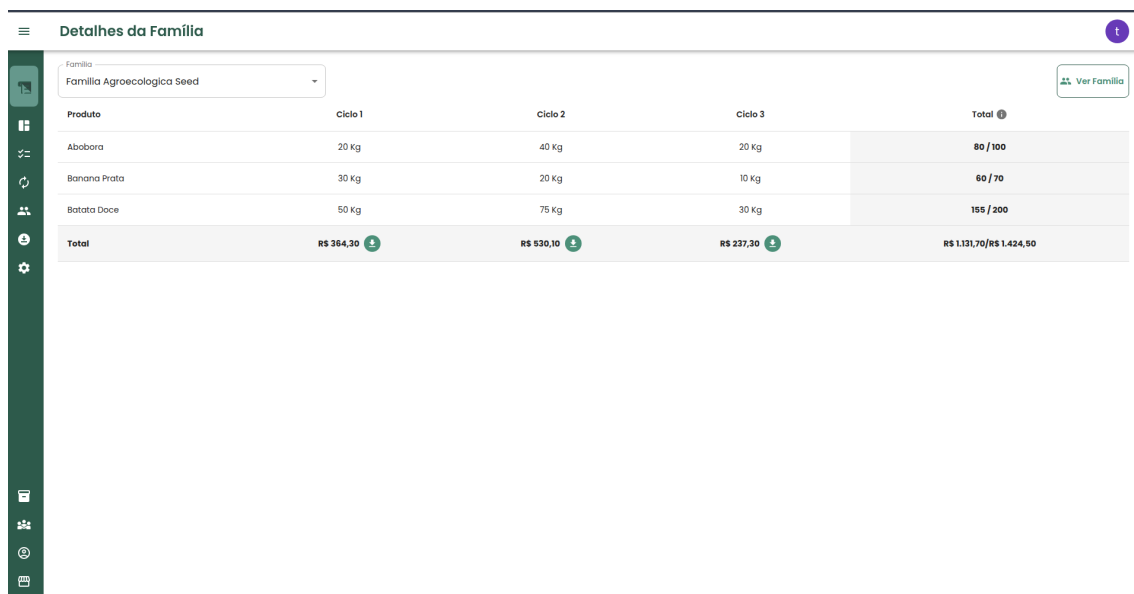


Figura 5.10: Relatórios

Com o objetivo de permitir o acompanhamento individual de cada família, o coordenador tem acesso à tela de detalhes da família. Nessa tela, os dados dos ciclos de entrega são organizados por família, permitindo visualizar quanto foi entregue em cada ciclo e o valor total recebido. Além disso, o sistema possibilita a geração do recibo referente àquele ciclo para a família selecionada.

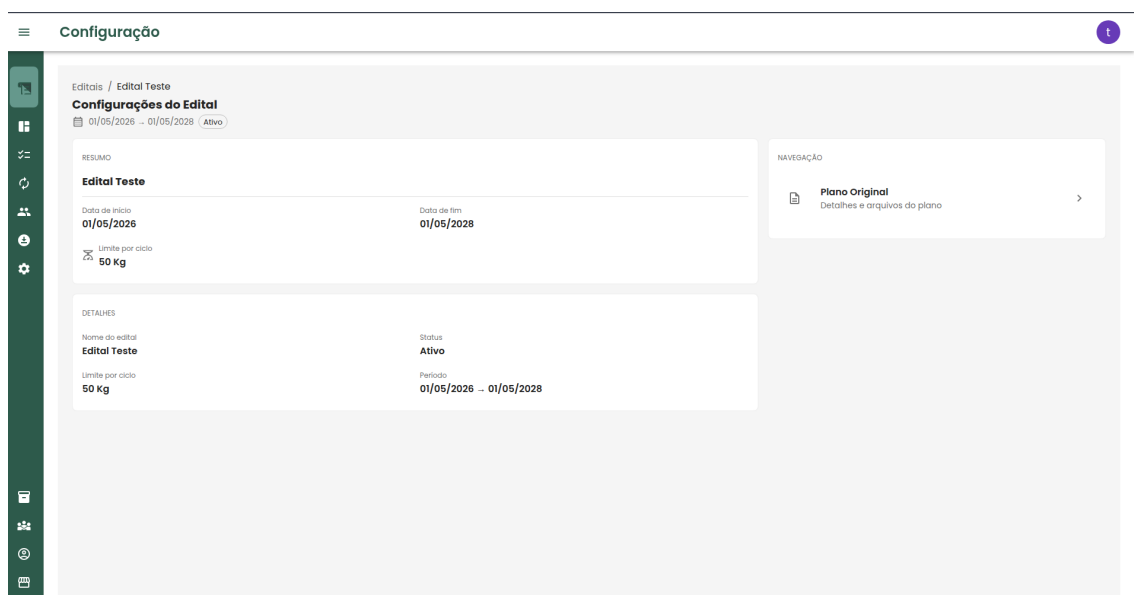
Na última coluna da tabela, há um recurso de apoio ao coordenador, que apresenta o acompanhamento acumulado das entregas ao longo dos ciclos. Essa coluna indica a quantidade de alimentos já entregue pela família e a quantidade ainda faltante em relação ao que foi definido no plano original. Na última célula da tabela, são apresentados o valor já recebido pela família e o valor que ainda deverá ser recebido até o final do edital. Abaixo, na Figura 5.11 segue um exemplo.



Produto	Ciclo 1	Ciclo 2	Ciclo 3	Total
Abóbora	20 Kg	40 Kg	20 Kg	80 / 100
Banana Prata	30 Kg	20 Kg	10 Kg	60 / 70
Batata Doce	50 Kg	75 Kg	30 Kg	155 / 200
Total	R\$ 384,30	R\$ 530,10	R\$ 237,30	R\$ 1.131,70 / R\$ 1.424,50

Figura 5.11: Detalhes da família

Para verificar informações gerais do edital, o sistema possui a tela de configuração, onde há um resumo do cadastro inicial, com informações de início e fim, assim como a quantidade limite de quilos de alimento por entrega e por fim, um card que ao ser clicado redireciona para o plano original. Abaixo segue a imagem com exemplo.



Configuração

Edital / Edital Teste

Configurações do Edital

01/05/2026 - 01/05/2028 (Ativo)

RESUMO

Edital Teste

Data de início: 01/05/2026

Data de fim: 01/05/2028

Limite por ciclo: 50 Kg

DETAHES

Nome do edital: Edital Teste

Status: Ativo

Limite por ciclo: 50 Kg

Período: 01/05/2026 - 01/05/2028

NAVEGAÇÃO

[Plano Original](#)

Detalhes e arquivos do plano

Figura 5.12: Configuração

Após a finalização do plano original, o dashboard passa a ser preenchido automaticamente com as informações necessárias para acompanhar o andamento do edital. A Figura 5.13 apresenta um exemplo da visualização dessa tela.

O dashboard é composto por indicadores que sintetizam o progresso. Entre essas informações, são apresentados os produtos cadastrados, a quantidade já entregue e a quantidade prevista no plano original. Ao interagir com um produto, por meio de um clique, o coordenador consegue visualizar a porcentagem de entrega correspondente a cada família, permitindo um acompanhamento mais detalhado da execução do edital.

Nessa mesma tela, também são apresentados dois gráficos relacionados ao andamento dos ciclos de entrega. O primeiro gráfico detalha quais produtos foram entregues em cada ciclo e suas respectivas quantidades. O segundo gráfico, localizado mais à direita, apresenta a quantidade total de alimentos entregue por ciclo, permitindo uma visão geral da evolução das entregas ao longo do edital.



Figura 5.13: Dashboard

Com o objetivo de atender a informações comuns a diferentes editais, o sistema possui entidades globais, representadas nas figuras a seguir. Essas entidades evitam a repetição desnecessária de dados.

Na Figura 5.14, são apresentados os produtos globais cadastrados no sistema.

Cada edital pode possuir sua própria representação desses produtos, o que permite utilizar uma mesma informação base, mas com valores e quantidades específicas para cada edital. Dessa forma, um mesmo produto pode estar presente em diferentes editais, porém com preço, quantidade prevista e demais configurações próprias de cada chamada.

Além dos produtos, o sistema também possui como entidades globais os coletivos e os destinos, representados, respectivamente, pelas figuras 5.15 e 5.17. Esses cadastros permitem organizar informações utilizadas em diferentes partes do sistema, facilitando o gerenciamento e a vinculação dessas entidades aos editais.

Por fim, há também o cadastro de usuários, representado na Figura 5.16. A partir dos usuários cadastrados, o sistema permite organizar os participantes e associá-los às famílias correspondentes, respeitando sua vinculação ao coletivo e à chamada pública correspondente.

Nome	
Abobora	 
Banana Prata	 
Batata Doce	 
Couve	 
Feijao Preto	 
Mandioca	 

Figura 5.14: Produtos

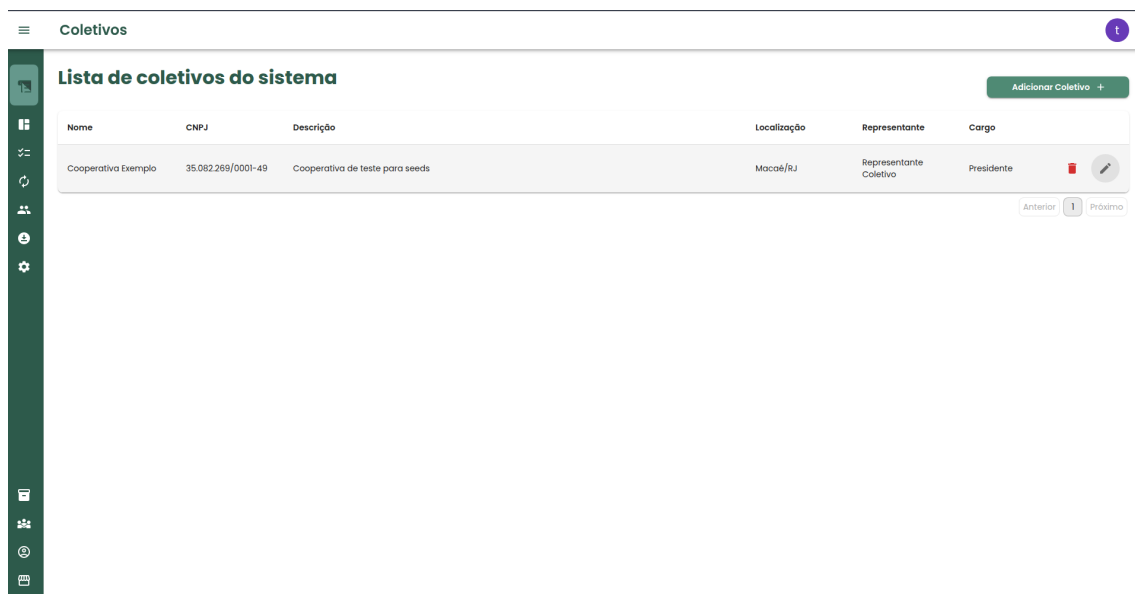


Figura 5.15: Coletivos

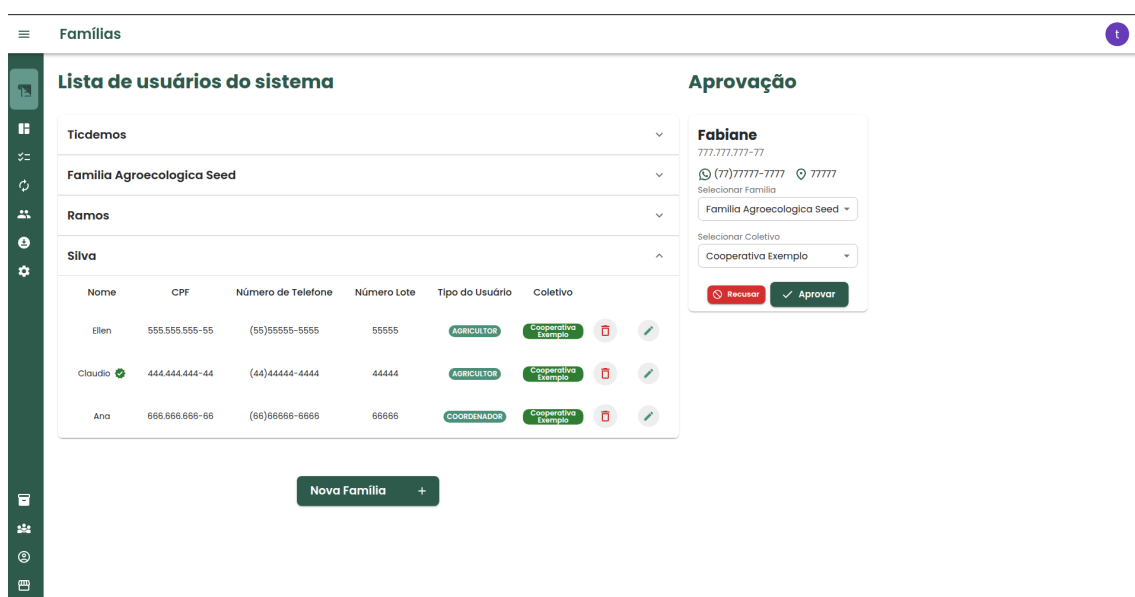


Figura 5.16: Usuários

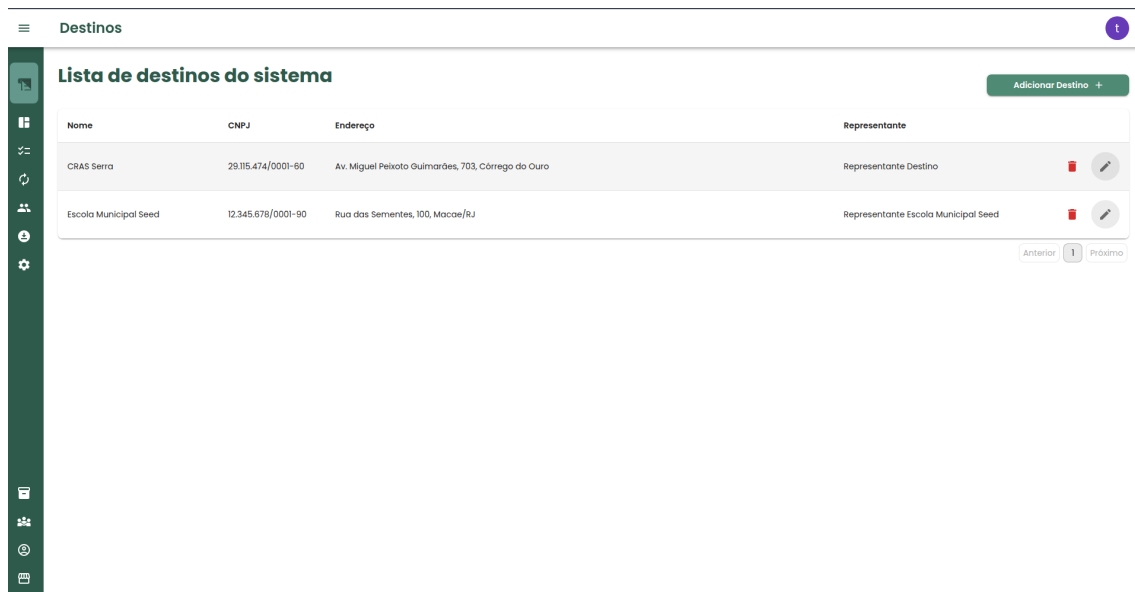


Figura 5.17: Destinos

5.5.2 Desafios de implementação do frontend

Assim como na seção dedicada aos desafios de implementação do backend, esta seção apresenta os principais desafios encontrados durante o desenvolvimento do frontend.

O primeiro desafio analisado corresponde à associação de famílias previamente cadastradas ao plano original de um edital em andamento. O plano original organiza os produtos em duas matrizes dinâmicas: uma por família e outra por destino. Nesse contexto, a interface precisa representar estruturas aninhadas recebidas da API, permitir sua edição e manter os dados sincronizados entre os componentes, o Redux e o backend. No trecho apresentado a seguir, destaca-se a implementação responsável por adicionar famílias ao plano original, verificando as associações existentes para evitar registros duplicados e atualizando o edital selecionado após a conclusão da requisição.

Código: 5.5: Async Thunk para adicionar famílias ao edital

```

1
2 export const adicionaFamiliaEdital = createAsyncThunk<
3   { familias: FamiliaProdutoEdital[]; idEdital: number },
4   number[] ,

```

```

5     { state: RootState }
6   >(
7     'edital/adicionaFamiliaEdital',
8     async (idFamilias: number[], { rejectWithValue, getState
9       }) => {
10       try {
11         const editalState = getState().edital
12         const familias = getState().familia.familias
13
14         const filteredFamilias: Familia[] = familias.
15           filter(
16             familia => idFamilias.includes(familia.id ??
17               0)
18           )
19
20         const existingNamesFamilias = new Set(
21           editalState.editalSelected?.planoOriginal.
22             familiasProdutoEdital
23             .map(familiaProdutoEdital =>
24               familiaProdutoEdital.familia?.nome ??
25               "") ?? []
26         );
27
28         const diffId = filteredFamilias
29           .filter(familia => !existingNamesFamilias.has
30             (familia?.nome ?? ""))
31           .map(familia => familia.id ?? 0);
32
33         const response = await api.post(
34           'v1/plano-original/${editalState.
35             editalSelected?.planoOriginal.id}/familia-
36             produtos',
37           diffId
38         )

```

```

30
31         return {
32             familias: response.data,
33             idEdital: editalState.editalSelected?.id ??
34                 -1
35         }
36     } catch (err) {
37         const error = err as AxiosError<ApiError>;
38         return rejectWithValue(
39             error.response?.data.message ?? 'Erro ao
40                 buscar as editais'
41         );
42     }
43 }
44 );

```

No trecho apresentado, é possível observar a função assíncrona responsável por selecionar as famílias informadas pelo usuário, evitar associações duplicadas e enviar ao backend apenas os novos registros.

A função foi criada por meio do `createAsyncThunk`, recurso utilizado do `redux` para executar operações assíncronas integradas ao estado global da aplicação. Como parâmetro de entrada, ela recebe uma lista contendo os identificadores das famílias selecionadas. Seu retorno contém as famílias associadas ao plano original e o identificador do edital que foi atualizado. A tipagem também permite o acesso ao estado global por meio da interface `RootState`.

Inicialmente, a função recupera do estado da aplicação o edital selecionado e a lista de famílias disponíveis. Em seguida, realiza uma filtragem para localizar apenas as famílias cujos identificadores estão presentes na lista recebida. Esse processo transforma os identificadores selecionados na interface em objetos que podem ser comparados com as famílias que já fazem parte do edital.

Após essa etapa, é criado um conjunto contendo os nomes das famílias previamente associadas ao plano original. O uso da estrutura `Set` permite verificar de forma direta se determinada família já está cadastrada. Com base nessa comparação, a função mantém apenas as famílias que ainda não possuem associação com o edital

e extrai seus identificadores para compor a requisição enviada ao backend.

Em seguida, os identificadores das novas famílias são enviados por meio de uma requisição HTTP do tipo POST. O endereço da requisição é construído utilizando o identificador do plano original pertencente ao edital selecionado. Após a conclusão da operação, a função retorna os dados recebidos do backend juntamente com o identificador do edital, possibilitando a atualização correta do estado da interface.

Caso ocorra uma falha durante o processo, o erro é convertido para o tipo `AxiosError<ApiError>`. A função utiliza `rejectWithValue` para retornar a mensagem fornecida pelo backend ou, caso ela não esteja disponível, uma mensagem padrão. Dessa forma, o erro pode ser tratado pelos estados de rejeição da operação assíncrona e posteriormente apresentado ao usuário. Destaca-se também como um desafio técnico o controle das entregas realizadas ao longo dos ciclos de execução. Nesse processo, o sistema não considera apenas os dados registrados no ciclo atual, pois o acompanhamento de cada produto depende do histórico de entregas acumulado até o período selecionado. Essa abordagem permite comparar a quantidade inicialmente planejada com o total efetivamente entregue e identificar o saldo que ainda deve ser fornecido.

Para realizar esse cálculo, a implementação percorre os ciclos anteriores e o ciclo atual, seleciona as entregas associadas à mesma família e acumula as quantidades registradas para o produto analisado. Além desse cálculo acumulado, o controle dos ciclos também envolve validação de datas, abertura e encerramento de períodos, estados de edição independentes para os itens e atualização de objetos aninhados no Redux, garantindo a consistência entre as informações apresentadas na interface e os dados armazenados no backend. No trecho a seguir, é apresentada a função responsável pelo cálculo da quantidade faltante de cada produto.

Código: 5.6: Função que calcula quanto falta para um agricultor entregar

```
1
2 const calculaEntregaFaltante = (
3     familia: Familia,
4     entregaItem: EntregaItem) => {
5
6     const entregasFamilia: Entrega[] = [];
```

```

7      const cicloAtual = ciclos.find(ciclo => ciclo.id ===
      cicloId);
8
9      if (!cicloAtual) return 0;
10
11     const previousCiclos = ciclos.filter(ciclo =>
12         dayjs(ciclo.dataFim).isSameOrBefore(dayjs(
13             cicloAtual.dataFim))
14     );
15
16     previousCiclos.forEach(ciclo => {
17         ciclo.entregas?.forEach(entrega => {
18             if (entrega.familia?.id === familia.id) {
19                 entregasFamilia.push(entrega);
20             }
21         })
22     });
23
24     const quantidadePlanejada = entregaItem.
25         produtoEditaIdTO.quantidade;
26     let quantidadeEntregueTotal = 0;
27
28     entregasFamilia.forEach(entrega => {
29         entrega.entregaItens.forEach(item => {
30             if (item.produtoEditaIdTO.id === entregaItem.
31                 produtoEditaIdTO.id) {
32                 quantidadeEntregueTotal += item.
33                     quantidadeEntregue ?? 0;
34             }
35         });
36     });
37
38     return quantidadePlanejada - quantidadeEntregueTotal;

```

Na função apresentada, vemos que o processo utiliza dados do ciclo atual e dos ciclos anteriores para comparar a quantidade planejada no plano original com o total já entregue até o período selecionado.

A função recebe como parâmetros a família responsável pela entrega e o item cujo saldo será calculado. Inicialmente, é criada uma lista para armazenar as entregas relacionadas à família. Em seguida, o ciclo atual é localizado por meio do identificador selecionado na interface. Caso nenhum ciclo correspondente seja encontrado, a função retorna o valor zero, evitando a execução do cálculo com dados inexistentes.

Após identificar o ciclo atual, são selecionados todos os ciclos cuja data de término seja anterior ou igual à data de término do ciclo selecionado. Para realizar essa comparação, é utilizada a biblioteca `dayjs`. Dessa forma, o cálculo considera não apenas as entregas realizadas no ciclo atual, mas também aquelas registradas nos ciclos anteriores.

Em seguida, a função percorre os ciclos selecionados e suas respectivas entregas. Durante esse processo, são mantidas apenas as entregas associadas à família recebida como parâmetro. Essa filtragem permite reunir o histórico de entregas da família até o ciclo atual, desconsiderando registros pertencentes a outras famílias.

A quantidade planejada é obtida a partir dos dados do produto associado ao item da entrega. Depois disso, todas as entregas selecionadas são percorridas novamente para localizar itens relacionados ao mesmo produto. Quando o produto é encontrado, sua quantidade entregue é adicionada ao total acumulado. Caso o valor esteja ausente, considera-se zero, impedindo que dados ainda não preenchidos comprometam o cálculo.

Ao final, a função subtrai a quantidade total entregue da quantidade planejada e retorna o saldo restante. Esse resultado pode ser utilizado pela interface para informar quanto ainda precisa ser entregue pela família em relação ao planejamento definido no edital.

Entre os recursos implementados no frontend, destaca-se o dashboard analítico responsável por acompanhar a execução do edital. A interface reúne informações provenientes do plano original e dos ciclos de entrega para apresentar a evolução

dos produtos ao longo do tempo. Nesse contexto, o sistema precisa organizar os dados por produto, família, destino e período, permitindo comparar as quantidades planejadas com aquelas efetivamente entregues. Essas informações são utilizadas na construção dinâmica de gráficos de barras, linhas e donut, oferecendo diferentes perspectivas sobre o andamento do edital. No trecho apresentado, a seguir, é definida a linha temporal utilizada pelos gráficos do dashboard. A implementação utiliza o `useMemo`, função do React, para calcular e armazenar o intervalo somente quando os ciclos ou as datas do edital forem alterados. Essa estratégia evita que o processamento seja repetido em todas as renderizações do componente, contribuindo para o desempenho da interface.

Código: 5.7: Constante que define a timeline dos gráficos do dashboard

```
1
2 const timeline = useMemo<TimelineData>(() => {
3     const start = dayjs(editalSelected?.dataInicio).startOf("
4         day");
5
6     const editalEnd = dayjs(editalSelected?.dataFim).startOf(
7         "day");
8
9     let lastCycleEnd: dayjs.Dayjs = dayjs(0);
10    ciclos.forEach((ciclo) => {
11        const cicloEnd = dayjs(ciclo.dataFim).startOf("day");
12        if (!cicloEnd.isValid()) {
13            return;
14        }
15        if (!lastCycleEnd || cicloEnd.isAfter(lastCycleEnd)) {
16            lastCycleEnd = cicloEnd;
17        }
18    });
19
20    const end = lastCycleEnd?.isBefore(editalEnd) ?
21        lastCycleEnd : editalEnd;
```

```

20     if (!start.isValid() || !end.isValid() || start.isAfter(
21         end, "day")) {
22         return { start: null, end: null, xLabels: [] };
23     }
24
25     const xLabels: string[] = [];
26     let cursor = start;
27     while (cursor.isBefore(end, "day") || cursor.isSame(end,
28         "day")) {
29         xLabels.push(cursor.format("YYYY-MM-DD"));
30         cursor = cursor.add(1, "day");
31     }
32
33     return { start, end, xLabels };
34 }, [ciclos, editalSelected?.dataInicio, editalSelected?.
35     dataFim]);

```

Analisando o código percebemos que a data de início do edital é utilizada como ponto inicial da linha temporal. Também é recuperada a data final prevista para o edital. Ambas são normalizadas para o início do dia por meio da biblioteca `dayjs`, evitando diferenças causadas por horários durante as comparações.

Em seguida, o código percorre todos os ciclos para localizar a data de término mais recente. Antes de utilizar cada data, a implementação verifica se ela é válida. Datas ausentes ou inválidas são desconsideradas, impedindo que comprometam a construção da série temporal. Ao encontrar um ciclo com término posterior ao último registrado, a variável `lastCycleEnd` é atualizada.

A data final da linha temporal é definida a partir da comparação entre o encerramento do último ciclo e o término do edital. Caso o último ciclo termine antes do edital, sua data é utilizada como limite. Caso ultrapasse o período previsto, prevalece a data final do edital. Dessa forma, os gráficos representam somente o período efetivamente alcançado pelos ciclos, sem ultrapassar os limites estabelecidos para a execução do edital.

Após a definição do intervalo, o código verifica se as datas são válidas e se o início não ocorre depois do término. Quando o período não pode ser construído,

são retornados valores nulos e uma lista vazia de rótulos. Esse tratamento evita a geração de gráficos com intervalos inconsistentes ou dados temporais incorretos.

Com o período validado, a implementação percorre todos os dias compreendidos entre as datas inicial e final. Cada data é formatada no padrão YYYY-MM-DD e adicionada à lista xLabels. O último dia também é incluído, permitindo que o eixo horizontal dos gráficos represente todo o intervalo analisado. Ao final, a função retorna a data inicial, a data final e os rótulos que serão utilizados na organização das entregas diárias.

Assim como nos demais desafios de implementação do frontend, a autenticação e a autorização exigem a integração entre a interface, o estado global da aplicação e os mecanismos de segurança implementados no backend. Esse processo não se limita à validação das credenciais durante o login, pois também envolve a manutenção e a renovação da sessão, a recuperação dos dados do usuário autenticado e a restauração do acesso quando a aplicação é recarregada.

Além da autenticação, a aplicação controla as funcionalidades disponíveis de acordo com o perfil do usuário. Os perfis de administrador, coordenador e agricultor possuem permissões e rotas iniciais distintas. Por esse motivo, a autorização é aplicada tanto na proteção das rotas quanto na validação do tipo de usuário e na filtragem das opções apresentadas no menu. Essa abordagem impede o acesso a páginas não autorizadas e adapta a interface às atividades que cada perfil pode executar.

No trecho apresentado a seguir, destaca-se a função assíncrona responsável por enviar as credenciais ao backend e tratar as possíveis respostas do processo de login. A implementação diferencia situações como credenciais inválidas, usuário ainda não liberado e falhas internas do servidor, permitindo que a interface apresente mensagens específicas para cada caso.

Código: 5.8: Função de login do sistema

```
1 export const login = createAsyncThunk(  
2   'auth/login',  
3   async (credentials: LoginCredentials, { rejectWithValue  
4     }) => {  
     try {
```

```

5      await api.post('v1/auth/login', credentials);
6  } catch (err) {
7      const error = err as AxiosError<ApiError>;
8      if (error.response?.status === 401) {
9          return rejectWithValue(
10             error.response?.data.message ?? '
              Credenciais invalidas. Verifique seu
              CPF e senha.'
11         );
12     }
13
14     if (error.response?.status === 403) {
15         return rejectWithValue(
16             error.response?.data.message ?? 'Usuario
              nao liberado. Contate o coordenador.'
17         );
18     }
19
20     if (error.response?.status === 500) {
21         return rejectWithValue(
22             error.response?.data.message ?? 'Erro
              interno no servidor. Tente novamente
              mais tarde.'
23         );
24     }
25
26     return rejectWithValue(
27         error.response?.data.message ?? 'Sistema fora
              do ar, por favor entre mais tarde.'
28     );
29 }
30 }
31 );

```

A função recebe um objeto do tipo LoginCredentials, contendo as informações ne-

cessárias para autenticação, como CPF e senha. Em seguida, realiza uma requisição HTTP do tipo POST para o endpoint `v1/auth/login`, enviando as credenciais no corpo da requisição. O uso de `await` faz com que a função aguarde a resposta do backend antes de finalizar a operação.

Quando a autenticação é concluída com sucesso, a função termina sem retornar dados. Nesse fluxo, o backend é responsável por adicionar os tokens de autenticação aos cookies da resposta. Portanto, o objetivo principal da requisição não é recuperar diretamente um token pelo corpo da resposta, mas permitir que o navegador armazene as credenciais de sessão configuradas pelo servidor. Posteriormente, outra operação pode consultar os dados do usuário autenticado e atualizar o estado da aplicação.

Caso ocorra uma falha, o erro capturado é convertido para o tipo `AxiosError<ApiError>`. Essa tipagem permite acessar de maneira estruturada o código de estado HTTP e a mensagem retornada pelo backend. A partir dessas informações, a função diferencia as principais situações de erro relacionadas ao processo de autenticação.

Quando o backend retorna o estado 401, a função considera que as credenciais fornecidas são inválidas. Nesse caso, utiliza a mensagem recebida da API ou, caso ela não esteja disponível, informa que o usuário deve verificar o CPF e a senha. Para o estado 403, a aplicação entende que as credenciais podem estar corretas, mas o cadastro ainda não foi liberado, orientando o usuário a entrar em contato com o coordenador.

O código também trata respostas com o estado 500, que representam falhas internas no servidor. Para essas situações, é retornada uma mensagem informando que ocorreu um erro no backend e que uma nova tentativa deve ser realizada posteriormente. Por fim, qualquer erro que não esteja entre os casos anteriores é tratado por uma mensagem genérica de indisponibilidade. Esse último fluxo também contempla falhas de conexão, interrupções de rede ou ausência de resposta do servidor. Em todas as situações de erro, é utilizado o `rejectWithValue`. Esse recurso permite que a mensagem seja encaminhada como conteúdo da ação rejeitada pelo Redux, facilitando sua utilização pelos redutores e componentes da interface. Dessa forma, o estado global pode registrar a falha e apresentar ao usuário uma mensagem cor-

respondente ao problema ocorrido.

5.6 Integração e Deploy

Durante o desenvolvimento do sistema, foi utilizado o Git como ferramenta de controle de versão, permitindo o acompanhamento das alterações realizadas no código-fonte e facilitando o trabalho colaborativo entre os desenvolvedores. Para o armazenamento remoto dos repositórios, foi utilizado o GitLab, que também oferece recursos de integração contínua e entrega contínua.

Com o objetivo de apoiar a integração contínua de novas funcionalidades e padronizar o processo de construção da aplicação, foi configurado, em cada projeto, um arquivo chamado `.gitlab-ci.yml`. Esse arquivo é responsável por definir as etapas executadas automaticamente pelo GitLab CI/CD, como a verificação do código, acionamento de testes automatizados, a execução do build e a geração de uma nova imagem Docker. No backend, esse processo utiliza o Maven para executar as etapas de teste e construção da aplicação. No frontend, o processo utiliza o Vite para gerar a versão de produção da interface. Neste primeiro momento a aplicação possui somente testes automatizados para pontos essenciais e comportamentos críticos do sistema. Após a criação da imagem, ela é publicada no Docker Hub, possibilitando que a versão mais recente da aplicação esteja disponível para implantação no ambiente de homologação.

O ambiente de homologação da aplicação encontra-se em uma VPS hospedada na Hostinger. Nesse ambiente, o deploy ainda é realizado de forma manual. Para isso, existe um script no servidor responsável por remover as imagens locais dos containers, baixar as versões mais recentes publicadas no Docker Hub e executar novamente o Docker Compose, fazendo com que os serviços da aplicação sejam atualizados e disponibilizados online.

Capítulo 6

Resultados

Atualmente, o sistema encontra-se em fase de homologação. Essa etapa tem como objetivo validar, junto aos usuários envolvidos no processo de gestão do PAA e ao Assentamento PDS Osvaldo de Oliveira, se as funcionalidades implementadas atendem às necessidades identificadas durante o levantamento de requisitos e se o fluxo do sistema corresponde à realidade operacional do programa.

A versão em homologação contempla as principais funcionalidades necessárias para a criação e o acompanhamento de um edital, permitindo organizar informações que antes eram controladas de forma manual, principalmente por meio de planilhas. Entre essas funcionalidades, destacam-se o cadastro de usuários, a aprovação de acesso ao sistema, o cadastro de famílias, a criação de editais e a definição dos perfis de acesso dos participantes.

Com isso, é possível observar que o sistema propõe uma nova forma de olhar para a gestão do PAA. Ao centralizar as informações em uma plataforma web, o acompanhamento dos editais tende a se tornar mais objetivo, claro e transparente. Além disso, a ferramenta reduz a dependência de controles manuais e facilita a consulta das informações pelos diferentes usuários envolvidos no processo.

Dessa forma, mesmo ainda estando em homologação, o sistema já demonstra potencial para contribuir com a organização da gestão do PAA no assentamento. A validação com os usuários será fundamental para identificar ajustes necessários, melhorar a experiência de uso e garantir que a ferramenta esteja adequada às demandas reais da comunidade antes de sua implantação definitiva.

Capítulo 7

Conclusão

7.1 Conclusão

O desenvolvimento deste trabalho permitiu a construção de um sistema web voltado ao apoio da gestão do Programa de Aquisição de Alimentos na modalidade Doação Simultânea, tendo como referência o contexto do Assentamento PDS Osvaldo de Oliveira. A proposta partiu de uma necessidade concreta: reduzir a dependência de controles manuais, centralizar informações da chamada pública e facilitar o acompanhamento das entregas realizadas pelas famílias agricultoras aos destinos vinculados ao edital. Ao longo do processo, o sistema deixou de ser apenas uma ideia inicial e passou a constituir uma aplicação em fase de homologação, com funcionalidades implementadas e com possibilidade de validação junto aos usuários envolvidos.

Entre os resultados alcançados, destacam-se a implementação de fluxos essenciais para o funcionamento da plataforma, funcionalidades que representam uma base importante para a gestão do PAA, pois permitem organizar informações que antes eram acompanhadas principalmente por planilhas e registros manuais. Do ponto de vista técnico, o trabalho atingiu um resultado relevante ao estruturar a aplicação em módulos separados, com contextos bem definidos. A escolha por uma arquitetura modular favoreceu a independência entre a interface e as regras de negócio, permitindo que novas funcionalidades fossem incorporadas de forma progressiva.

Além da implementação, o projeto evidenciou a importância de aproximar o desenvolvimento de software do contexto social em que a ferramenta será utilizada.

O levantamento de requisitos, as reuniões com os gestores do PAA e os ajustes realizados a partir de novas demandas evidenciam que a construção de uma solução não depende apenas da escolha de tecnologias, mas também da compreensão do processo real de trabalho, agregando valor o que foi desenvolvido.

Os trabalhos futuros devem partir da análise do que já foi implementado e do estágio atual do sistema. Para as novas implementações, a metodologia participativa deverá continuar presente, para que o software tenha maior aderência aos usuários e faça diferença no dia a dia dos coordenadores, administradores e agricultores. Ao analisarmos a arquitetura que foi planejada e implementada, observamos que ela atende a uma demanda simples. No entanto, se pensarmos em um contexto mais amplo, considerando o Brasil, o sistema deverá implementar métricas de desempenho, reforçar a segurança dos dados, dividir funcionalidades em serviços e contar com uma estrutura capaz de suportar milhares de requisições, caso seja necessário.

Por fim, conclui-se que este trabalho contribui tanto para a organização técnica de uma aplicação web quanto para a construção de uma ferramenta voltada a uma demanda social específica. O sistema desenvolvido oferece uma base para tornar a gestão das chamadas públicas mais transparente, menos dependente de processos manuais e mais próxima das necessidades dos usuários. Ao mesmo tempo, sua continuidade exigirá novas etapas de amadurecimento, especialmente para que a solução possa deixar de ser apenas um sistema em homologação e se tornar uma ferramenta estável.

Referências Bibliográficas

- SOMMERVILLE, I. *Software Engineering*. 10 ed. Boston, Pearson, 2016.
- Washizaki, H. (Ed.). *Guide to the Software Engineering Body of Knowledge (SWE-BOK Guide), Version 4.0*. Los Alamitos, CA, IEEE Computer Society, 2024.
- ACM, IEEE COMPUTER SOCIETY. *Software Engineering 2014: Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering*. ACM and IEEE Computer Society, 2015.
- PERRY, D. E., PORTER, A. A., VOTTA, L. G. “Empirical Studies of Software Engineering: A Roadmap”. In: *Proceedings of the Conference on The Future of Software Engineering*, pp. 345–355. ACM, 2000.
- ROYCE, W. W. “Managing the Development of Large Software Systems”. In: *Proceedings of IEEE WESCON*, 1970.
- BOEHM, B. W. “A Spiral Model of Software Development and Enhancement”, *Computer*, v. 21, n. 5, pp. 61–72, 1988. doi: 10.1109/2.59.
- SCACCHI, W. “Process Models in Software Engineering”. In: Marciniak, J. J. (Ed.), *Encyclopedia of Software Engineering*, John Wiley & Sons, New York, 2002.
- RALPH, P. “Software Engineering Process Theory: A Multi-Method Comparison of Sensemaking–Coevolution–Implementation Theory and Function–Behavior–Structure Theory”, *Information and Software Technology*, v. 70, pp. 232–250, 2016. doi: 10.1016/j.infsof.2015.06.010.
- BECK, K., BEEDLE, M., VAN BENNEKUM, A., et al. *Manifesto for Agile Software Development*. Agile Alliance, 2001. Disponível em: <<https://agilemanifesto.org/>>. Acesso em: 28 mar. 2026.

- DYBÅ, T., DINGSØYR, T. “Empirical Studies of Agile Software Development: A Systematic Review”, *Information and Software Technology*, v. 50, n. 9–10, pp. 833–859, 2008. doi: 10.1016/j.infsof.2008.01.006.
- KUHRMANN, M., TELL, P., HEBIG, R., et al. “What Makes Agile Software Development Agile?” *IEEE Transactions on Software Engineering*, v. 48, n. 9, pp. 3523–3539, 2022. doi: 10.1109/TSE.2021.3099532.
- ISO/IEC 25010:2023 – *Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*. International Organization for Standardization, 2023. Disponível em: <<https://www.iso.org/standard/78176.html>>. Acesso em: 28 mar. 2026.
- SILVEIRA, S. A. D. *Software livre: a luta pela liberdade do conhecimento*. São Paulo, Fundação Perseu Abramo, 2004.
- GNU PROJECT. *What Is Free Software?* Free Software Foundation, 2026. Disponível em: <<https://www.gnu.org/philosophy/free-sw.en.html>>. Acesso em: 28 mar. 2026.
- FREE SOFTWARE FOUNDATION. *What Is Free Software and Why Is It So Important for Society?* Free Software Foundation, 2026. Disponível em: <<https://www.fsf.org/about/what-is-free-software>>. Acesso em: 28 mar. 2026.
- OPEN SOURCE INITIATIVE. *History of the Open Source Initiative*. Open Source Initiative, 2026. Disponível em: <<https://opensource.org/about/history-of-the-open-source-initiative>>. Acesso em: 28 mar. 2026.
- MOCKUS, A., FIELDING, R. T., HERBSLEB, J. D. “A Case Study of Open Source Software Development: The Apache Server”. In: *Proceedings of the 22nd International Conference on Software Engineering*, pp. 263–272, Limerick, Ireland, 2000. ACM.
- SCOTTO, M., SILLITTI, A., SUCCI, G. “An Empirical Analysis of the Open Source Development Process Based on Mining of Source Code Repositories”, *International Journal of Software Engineering and Knowledge Engineering*, v. 17, n. 2, pp. 231–247, 2007.

LONCHAMP, J. “Open Source Software Development Process Modeling”. In: Acuña, S. T., Juristo, N. (Eds.), *Software Process Modeling*, v. 10, *International Series in Software Engineering*, Springer, pp. 29–64, Boston, 2005. doi: 10.1007/0-387-24262-7_2.

MACCORMACK, A., RUSNAK, J., BALDWIN, C. Y. “Exploring the Structure of Complex Software Designs: An Empirical Study of Open Source and Proprietary Code”, *Management Science*, v. 52, n. 7, pp. 1015–1030, 2006. doi: 10.1287/mnsc.1060.0552.

Spring Boot. Spring, 2026. Disponível em: <<https://spring.io/projects/spring-boot>>. Acesso em: 28 mar. 2026.

PostgreSQL: About. PostgreSQL Global Development Group, 2026. Disponível em: <<https://www.postgresql.org/about/>>. Acesso em: 28 mar. 2026.

React Documentation. React, 2026. Disponível em: <<https://react.dev/>>. Acesso em: 28 mar. 2026.

CHACON, S., STRAUB, B. *Pro Git*. 2 ed. Berkeley, CA, Apress, 2014. Disponível em: <<https://git-scm.com/book/en/v2>>.

Why GitLab? GitLab, 2026a. Disponível em: <<https://about.gitlab.com/why-gitlab/>>. Acesso em: 28 mar. 2026.

Get Started with GitLab CI/CD. GitLab, 2026b. Disponível em: <<https://docs.gitlab.com/ci/>>. Acesso em: 28 mar. 2026.

What Is Docker? Docker, 2026a. Disponível em: <<https://docs.docker.com/get-started/docker-overview/>>. Acesso em: 28 mar. 2026.

Docker Hub. Docker, 2026b. Disponível em: <<https://docs.docker.com/docker-hub/>>. Acesso em: 28 mar. 2026.